

How to prove more false statements: Fiat–Shamir limitations on (generated) R1CS

Giacomo Fenzi
`giacomo.fenzi@epfl.ch`
EPFL

September 9, 2026

Abstract

The Fiat–Shamir (FS) transformation is a technique that converts interactive protocols into non-interactive ones. FS is secure in idealized models such as the random oracle model (ROM) (if the interactive protocol satisfies a condition known as state-restoration soundness).

It is known that there are protocols whose FS transformation is secure in the ROM, yet insecure when instantiated with any concrete hash function. Historically, these protocols were contrived (as in, they were designed so their FS transformation would be unsound). Khovratovich, Rothblum and Soukhanov (CRYPTO 2025) showed that a class of natural (and practically deployed) protocols based on a protocol of Goldwasser, Kalai and Rothblum (JACM 2015) was also unsound when compiled with FS and any concrete hash function.

We extend the attack to a different class of protocols: those whose instances are generated by running a program.

This setting covers concrete trends in modern proof systems, in which the computation to be proven is described by a program (often adversarially generated) which is then either compiled or autonomously converted into an instance of target relation such as rank-1 constraint satisfaction (R1CS). We show that, when the conversion process is “expressive enough”, an adversary controlling the program code can break soundness of the non-interactive proof system.

The attacks generalize to a wide class of protocols: any protocol in which a cheating prover can prepare an accepting transcript before the statement is bound. We show that variants of the Spartan (CRYPTO 2020) and Aurora (EUROCRYPT 2019) proof systems for R1CS fall in this class.

Complementing the attacks, we formalize a mitigation: deriving the Fiat–Shamir challenges from the generated statement, rather than from the program that generates it, provably reduces the soundness of the compiled protocol to that of the underlying protocol for the non-generated relation.

Keywords: succinct arguments; Fiat–Shamir; diagonalization

1 Introduction

Succinct non-interactive arguments (SNARGs) [Mic00] have emerged as a fundamental cryptographic primitive. They allow a prover to certify the correct execution of a computation with a proof that can be exponentially smaller, and substantially faster to verify, than the computation itself. SNARGs now support applications ranging from distributed computation to digital identity systems.

Fiat–Shamir transformation. A common route to concretely efficient SNARGs, including the families studied here, is to apply the celebrated Fiat–Shamir transformation [FS86] to a public-coin interactive proof (or argument). The transformation removes interaction by replacing each verifier message with a hash of the transcript so far. Intuitively, the transformation is sound since a prover should be unable to predict the verifier message without hashing the preceding transcript. The intuition can be formalized in the random oracle model (ROM) [BR93], where the hash function is modeled as an ideal random function: on each previously unqueried input, it returns an independent, uniformly distributed string in $\{0, 1\}^\lambda$. Under suitable conditions on the interactive protocol, Fiat–Shamir yields a secure SNARG in the ROM [PS96; BCS16; CCHLR18].

The ROM, however, is an idealized model: protocols that are secure in the ROM might not be secure when the random oracle is instantiated with a concrete hash function.

We seek to understand when Fiat–Shamir remains secure after replacing the random oracle with a concrete hash function.

Limitations of Fiat–Shamir. Prior work constructed protocols whose Fiat–Shamir compilations are secure in the ROM but insecure for every concrete choice of hash function [Bar01; GK03; CGH04]. More precisely, these protocols are *adaptively unsound*: once a concrete hash function is chosen, a malicious prover can prove a false statement (both the statement and the proof may depend on the hash choice). The protocols refuted the conjecture that any protocol that is secure in the ROM after Fiat–Shamir is secure when the ROM is instantiated with a sufficiently good hash function. However, the counterexample protocols (and subsequent work [BBHMR19; BHK26]) were specifically designed to refute the conjecture and thus were relatively contrived, which led to a revised informal conjecture that any “natural” protocol that after compilation is sound in the ROM remains sound in the plain model for an appropriate hash function.

Khovratovich, Rothblum, and Soukhanov [KRS25] refuted this revised conjecture, showing that a natural protocol combining the GKR protocol [GKR15] with a polynomial commitment scheme [KZG10] was insecure after Fiat–Shamir compilation with any concrete hash function.

The counterexample exploited the (desirable) property of GKR that it need not commit to the internal wires of the computation being proved. A malicious prover can exploit this flexibility to construct a circuit that internally computes its own first Fiat–Shamir challenge and then uses the knowledge of the challenge to violate soundness. The attack affected concretely deployed systems [Pol24] and prompted work on mitigations [AY25].

Since the result of [KRS25] relied crucially on the structure of the GKR protocol (namely on the property that it does not commit to the internal wires), one might wonder the following:

To what extent do [KRS25]-style counterexamples to Fiat–Shamir rely on the structure of the protocol being attacked?

1.1 Our results

In this work, we continue the study of counterexamples to the Fiat–Shamir heuristic. We write $\text{FS}^h(\text{IP})$ for the compilation of an interactive protocol IP with a hash function h . From prior work, we know that as long as IP is state-restoration sound (resp. state-restoration knowledge sound) and h is a random oracle then $\text{FS}^h(\text{IP})$ is an adaptively sound (resp. knowledge-sound) non-interactive argument in the ROM (see, e.g., [CY24]).

Theorem 1 (Informal). *There exists a relation $\mathcal{R}$ such that $\text{FS}^h(\text{IP})$ is not adaptively sound for any “natural” IP for $\mathcal{R}$ and any sufficiently efficient hash function h .*

In Section 1.2 we elaborate on the details of Theorem 1 (including the definition of naturality, and the relation between the complexity of h and the naturality of IP).

Concrete instantiation. We define *generated R1CS* $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$, which is a variant of the rank-1-constraint satisfaction (R1CS) relation [GGPR13] that fulfils the promise of Theorem 1: $\text{FS}^h(\text{IP})$ is not adaptively sound for any IP for $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$ that satisfies the naturality condition and concrete hash function h .

We show that the compilation of two popular interactive oracle proofs (IOPs) [BCS16; RRR16] for R1CS (namely the univariate IOP of [BCRSVW19] and the multilinear IOP of [Set20]) yields protocols that are natural in the sense of Theorem 1. Letting $\text{BCS}^h(\text{IOP})$ denote the compilation of IOP using a hash function h , since these IOPs are round-by-round knowledge sound, we obtain the following corollary.

Corollary 1 (Informal). *There exists an IOP for generated R1CS $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$ such that:*

- *if h is a random oracle, $\text{BCS}^h(\text{IOP})$ is adaptively knowledge sound in the ROM.*
- *for any concretely efficient¹ hash function h , $\text{BCS}^h(\text{IOP})$ is not adaptively sound.*

1.2 Technical overview

As in prior work, our techniques to attack the Fiat–Shamir heuristic rely on *diagonalization*. Our aim will be for the adversary to construct an instance that “knows its own Fiat–Shamir challenge”, and use this to attack soundness. We first recall the *adaptive* Fiat–Shamir transformation.

Construction 1 (Informal). *Let $\mathbf{V}$ be a k -round interactive verifier for a relation $\mathcal{R}$ and h a hash function. The adaptive Fiat–Shamir transformation yields the following verifier:*

$$\begin{aligned}
 &\mathbf{V}_{\text{ARG}}^h(\mathbf{x}, \pi): \\
 &\quad - \text{Parse } \pi := (m_1, \dots, m_k). \\
 &\quad - \text{For } i = 1, \dots, k, \text{ let} \\
 &\qquad \qquad \qquad \rho_i := h(i, \mathbf{x}, m_1, \dots, m_i) . \\
 &\quad - \text{The verifier checks that } \mathbf{V}(\mathbf{x}, (m_i, \rho_i)_{i \in [k]}) = 1.
 \end{aligned} \tag{1}$$

Note that in Equation 1 $\mathbf{x}$ is included in the hash computation for the first round. This is necessary and sufficient for the Fiat–Shamir transformation to be *adaptively sound* in the ROM;

¹We elaborate on the efficiency requirements in Remark 1.

without it, the transformation only achieves *non-adaptive* soundness.² Indeed, an adversary can (for many proof systems) choose an invalid instance $\mathfrak{x}$ depending on the concrete hash function h and produce an accepting proof for it.

Our observation is that, even if $\mathfrak{x}$ is included in Equation 1, we can construct relations where the instance $\mathfrak{x}$ can predict its own hash output.

Given any relation $\mathcal{R}$, one can construct a *generated* version of the relation (we give the general definition in Section 4); in this overview we take circuit satisfiability (CSAT) for concreteness.

Definition 1 (Informal). *Let $c > 1$ be an arbitrary constant³. The **c-generated CSAT** relation is the relation consisting of $\mathfrak{x} = (1^T, 1^n, 1^m, s)$, $\mathfrak{w} = \mathbf{x} \in \{0, 1\}^n$ such that:*

- *When s is interpreted as a deterministic program, it runs in at most T^c steps and generates a circuit $\varphi: \{0, 1\}^n \rightarrow \{0, 1\}$ of size $|\varphi| \leq m$; and*
- *$\varphi(\mathbf{x}) = 1$.*

Generated relations have a peculiar feature: one can diagonalize the instance using a version of Kleene’s second recursion theorem [Kle52].

Claim 1 (Informal). *In a standard programming model, for every program F there is an efficiently computable program s^* such that executing s^* yields $F(s^*)$. Further, $|s^*| = O(|F|)$ and constructing it takes time $\text{poly}(|F|)$. If $F(s^*)$ can be computed in T steps, then executing s^* takes $(T + |F|) \cdot \text{poly}(|F|, \log T)$ steps.*

Concretely, given an arbitrary program F , an instance $\mathfrak{x}$ of the generated relation can contain a program s^* that computes $F(s^*)$: s^* computes an arbitrary function of *its own code*. In our attack, one can let F be the program that computes a prover message m_1 and the Fiat–Shamir hash $\rho_1 = h(1, \mathfrak{x}, m_1)$. In the context of c -generated CSAT, one can then produce a circuit that contains the verifier randomness ρ_1 that the verifier of the Fiat–Shamir transformation would recompute.

Many PCPs, IOPs and interactive arguments for c -generated CSAT have the following two properties.

Definition 2 (Informal). *An interactive k -round verifier $\mathbf{V}$ for c -generated CSAT is **semantic** if its decision depends on the instance only through the generated statement: there is a verifier $\overline{\mathbf{V}}$ such that $\mathbf{V}(\mathfrak{x}, \cdot) = \overline{\mathbf{V}}(\varphi, \cdot)$ whenever the program in $\mathfrak{x}$ generates φ .*

*A semantic verifier is **fooled by delayed statement choice** if there exist deterministic efficient algorithms $(\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k, \text{Bind})$ such that, for every instance $\mathfrak{x}$ and every challenge sequence $\rho_1, \dots, \rho_k$, setting $m_i := \tilde{\mathbf{P}}_i(\mathfrak{x}, \rho_1, \dots, \rho_{i-1})$, the circuit $\varphi := \text{Bind}(\mathfrak{x}, \rho_1, \dots, \rho_k)$ is unsatisfiable and yet*

$$\overline{\mathbf{V}}(\varphi, (m_i, \rho_i)_{i \in [k]}) = 1.$$

In words: the prover chooses its messages round by round, and only after seeing all of the verifier’s randomness is a false statement chosen under which the transcript verifies. In our setting, this suffices: the instance commits only to a program, and by Claim 1 a program can compute its own transcript (whose challenges are determined by hashing) and generate the formula afterwards. This leads to our main theorem: such a protocol is insecure when Fiat–Shamir is instantiated with any sufficiently efficient hash function.

²Omitting the statement is an instance of the “weak Fiat–Shamir”, which has led to concretely vulnerable implementations. See [DMWG23] for a survey.

³The constant c regulates how long the generator program can run. Depending on the complexity of the diagonalization and the hash function used, larger values of c may be used.

Theorem 2. *If $\mathbf{V}$ is a semantic verifier for $\mathbf{c}$ -generated CSAT that is fooled by delayed statement choice, then $\mathbf{V}_{\text{ARG}}^h$ (as in Construction 1) is not adaptively sound for any sufficiently efficient hash function h .*

Sketch. Let $(\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k, \text{Bind})$ be as in Definition 2. For each candidate T , choose supported parameters $n, m \leq T$, write $\mathbb{x}_{T,s} := (1^T, 1^n, 1^m, s)$, and let F_T be the program that, on input s , computes $m_i := \tilde{\mathbf{P}}_i(\mathbb{x}_{T,s}, \rho_1, \dots, \rho_{i-1})$ and ρ_i as in (1) for $i = 1, \dots, k$, and outputs

$$F_T(s) := \text{Bind}(\mathbb{x}_{T,s}, \rho_1, \dots, \rho_k).$$

By Claim 1, we can construct s_T^* that outputs $F_T(s_T^*)$.

We now choose T . Let K be the total code length of h and the malicious algorithms. Since $|F_T| = O(K + \log T)$, Claim 1 gives constants C, k and an exponent d such that (letting $N := |\mathbb{x}_{T,s_T^*}|$):

$$|s_T^*| \leq C(K + \log T), \quad \text{time}(s_T^*) \leq CN^d(K + \log N)^k.$$

As long as $d < \mathbf{c}$, one can choose a power of two T large enough that $|s_T^*| \leq T$ and $C(4T)^d(K + \log(4T))^k \leq T^{\mathbf{c}}$. This T can be efficiently found by doubling. Then, $N \leq 4T$, so s_T^* respects the time budget. Fix this T and set $\mathbb{x}^* := \mathbb{x}_{T,s_T^*}$.

Run $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k$ on $\mathbb{x}^*$, deriving each challenge using (1), and set

$$\varphi^* := \text{Bind}(\mathbb{x}^*, \rho_1, \dots, \rho_k).$$

By construction, the program in $\mathbb{x}^*$ generates $\varphi^* = F_T(s_T^*)$; since φ^* is unsatisfiable, $\mathbb{x}^*$ is false. The Fiat–Shamir verifier recomputes exactly these challenges, and since $\mathbf{V}$ is semantic, $\mathbf{V}(\mathbb{x}^*, \cdot) = \bar{\mathbf{V}}(\varphi^*, \cdot)$, which accepts the transcript by Definition 2. Hence $\mathbb{x}^*$ is false but the verifier accepts: $\mathbf{V}_{\text{ARG}}^h(\mathbb{x}^*, (m_1, \dots, m_k)) = 1$. $\square$

Remark 1. *The “sufficiently efficient” in Theorem 2 refers to the relation between the budget $\mathbf{c}$ in the $\mathbf{c}$ -generated CSAT relation and the exponent d in the running time of s_T^* . This latter exponent depends on the complexity of the hash function h , the efficiency of the malicious algorithms in the diagonalization and the complexity of constructing the fixed point. The attack succeeds as long as $\mathbf{c} > d$.*

Remark 2. *Definition 2 captures a wide class of protocols for generated relations. We are not aware of natural proof systems that avoid it, other than trivial ones (such as the ones where the prover sends the NP-witness and the verifier checks membership in the relation) or ones in which the generation of the instance from the program is also itself proven (instead of computed).*

Concrete implications. In Sections 6 and 7 we show that two round-by-round knowledge sound IOP families for the generated R1CS relation are semantic and fooled by delayed statement choice in the sense of Definition 2. The IOPs are (trivial) adaptations of existing IOPs for R1CS, where additionally the prover and verifier run the generating program s to derive the R1CS matrices. Theorem 2 straightforwardly adapts, which proves Theorem 1: there exist IOPs for generated R1CS that are insecure when compiled via the BCS transformation with any sufficiently efficient hash function h . While generated R1CS may seem like a contrived relation, it is somewhat close to practically relevant designs of proof systems for R1CS. In modern proof systems, the user wants

to prove a computation, say $s(w) = 0$, in some computation model. The code s of the computation being proven is then interpreted or compiled into corresponding R1CS matrices. When s is untrusted, it is hard to guarantee that the compilation process cannot be (possibly undetectably) influenced to generate instances that satisfy the preconditions for our attacks. In that case, if the Fiat–Shamir hash contains only s (or a commitment to s in the preprocessing case) the protocol may be vulnerable. Luckily, there are mitigations that can be taken.

- If the R1CS matrices are used as the statement identifier in Equation 1, then the attacks that we describe do not apply. In fact, hashing the generated statement provably reduces adaptive soundness to that of the underlying non-generated protocol (Theorem 5.5).
- The attack also requires the adversary to specify a chosen computation s to be proven. We recommend caution and careful audits in any setting in which such computations are fully or partially under the control of third parties (e.g. when compiling user code, automatic precompiles, AI-generated circuits, and similar).
- [AY25] describes a variant of the Fiat–Shamir called Extended Fiat–Shamir (XFS), designed to prevent the diagonalization that this paper and [KRS25] employ. XFS prevents the attacks in this paper, at the cost of a more expensive verifier. To our knowledge, the security proofs in [AY25] do not give provable guarantees that XFS prevents variants of the attacks in the paper.

While our concrete IOP attacks target R1CS, the generated-relation construction of Section 4 applies to *any* relation: for every relation $\mathcal{R}$ and every budget exponent c , the c -generated variant $\mathcal{R}[c]$ has a self-referential semantic scheme. We thus believe our attacks apply to other relations and designs, such as proof systems for virtual machines, as long as the relation is (partially or fully) generated. Other transformations such as [Fis05] are similarly vulnerable.

Theoretical implications. Our results give limitations on showing security guarantees for interactive protocols compiled with Fiat–Shamir in the plain model. We showed that any natural protocol (in the sense of Definition 2 and definition 3.3) for generated relations (Definition 3.2) is insecure when compiled via Fiat–Shamir instantiated with a sufficiently efficient hash function h . Positive results on the vanilla Fiat–Shamir transformation must therefore circumvent at least one of these caveats. For a protocol to be secure after Fiat–Shamir in the plain model: either the protocol must not be natural (which seems to require deviating from common techniques in building interactive proof systems), or the relation must not be generated (which rules out generic statements that handle every NP-relation) or the hash function must be inefficient (which is, generally, undesirable). We believe understanding the limitations of Fiat–Shamir and Fiat–Shamir-like transformations to be a central stepping stone for proving future positive results.

1.3 Related work

Closely related to our work is [KRS25, Section 4]. A strawman mitigation to the diagonalization attacks that they present would be inspecting the GKR circuit proven to ensure that it does not contain an implementation of the FS hash function or the commitment scheme employed. In [KRS25, Section 4], the authors show that this mitigation is insufficient, by using universal computation and quines (which also appear in our techniques). [KRS25, Section 4.1] chooses the GKR circuit to interpret the witness as a program, and then chooses the witness to be a self-referential program. The attack is impractical as the malicious prover has to prove a universal

machine execution through GKR. [KRS25, Section 4.2] extends this attack by using commitment decomposition to (for specific commitments) to avoid universal machine simulation.

Our work uses related self-reference techniques, but places the self-referential program in the generation of the statement, rather than in the GKR circuit. This has the advantage that it imposes minimal requirements on the proof system used: our attack extends to non-GKR based protocol such as variants of Spartan and Aurora over generated R1CS.

2 Preliminaries

We use the following conventions and notation:

- We write $[N] := \{1, \dots, N\}$. Coordinates of an N -entry vector are indexed by $0, \dots, N-1$ unless stated otherwise.
- Logarithms are base 2 unless stated otherwise.
- We denote by $\mathbf{e}_0 = (1, 0, \dots, 0)$ the unit vector, whose dimension is clear from context.
- The “hat” symbol over a function (e.g. $\hat{p}$) denotes a polynomial.
- We interchangeably interpret vectors $\mathbf{f} \in \Sigma^\ell$ as functions $f: [\ell] \rightarrow \Sigma$.
- For a field $\mathbb{F}$, number of variables m , and individual degree bound d , let $\mathbb{F}^{<d}[X_1, \dots, X_m]$ be the polynomials over $\mathbb{F}$ whose degree in each variable is at most $d-1$.
- The notation $x \leftarrow S$ means that x is sampled uniformly from S . If A is randomized, $x \leftarrow A$ means that x has the output distribution of A .

In the paper, we say an algorithm is *efficient* if it runs in (probabilistic) polynomial time. Where needed, we track the exponent using the following code-sensitive definition.

Definition 2.1. Fix the programming system used below. For $\theta \geq 1$, a program F is **θ -tame** if it halts within $(|F| + 1)(n + 1)^\theta$ steps on every input of length n . A coded function (f, F) is θ -tame if F computes f and satisfies this bound. We write $\mathcal{F}_\theta$ for the class of these coded functions.

Fixed constants can be absorbed by padding the program code. Thus, for a fixed program, this convention is equivalent to the usual $O(n^\theta)$ bound.

2.1 Error-correcting codes

The (fractional) Hamming distance between two vectors is the fraction of positions at which they differ.

Definition 2.2. Let $f, g: S \rightarrow \Sigma$. The **(fractional) Hamming distance** between f and g is

$$\Delta(f, g) = \Pr_{i \leftarrow S}[f(i) \neq g(i)].$$

For a non-empty set $\mathcal{C}$, we write $\Delta(f, \mathcal{C}) = \min_{g \in \mathcal{C}} \Delta(f, g)$.

Basic definitions. We recall the definition of error-correcting codes.

Definition 2.3. An **error-correcting code** $\mathcal{C}$ over an alphabet Σ with block length ℓ is a subset $\mathcal{C} \subseteq \Sigma^\ell$. Its **minimum distance** and **rate** are

$$\delta(\mathcal{C}) := \min_{\substack{f, g \in \mathcal{C} \\ f \neq g}} \Delta(f, g), \quad \rho(\mathcal{C}) := \frac{\log_{|\Sigma|} |\mathcal{C}|}{\ell}.$$

When an injective encoder $\mathcal{C}: \Sigma^k \rightarrow \Sigma^\ell$ is specified, we also use $\mathcal{C}$ for its image. We recall the definition of list-decoding.

Definition 2.4. Let $\mathcal{C}$ be an error-correcting code over an alphabet Σ with block length ℓ . For any $f: [\ell] \rightarrow \Sigma$ and $\delta \in (0, 1)$, we define the **list around f at distance δ** as

$$\Lambda(\mathcal{C}, \delta, f) := \{g \in \mathcal{C} \mid \Delta(f, g) \leq \delta\}.$$

Further, we write $|\Lambda(\mathcal{C}, \delta)| := \max_f |\Lambda(\mathcal{C}, \delta, f)|$.

2.2 Interactive oracle proofs

Relations. For IOPPs in this subsection, we represent a relation by triples $(\mathbf{x}, \mathbf{y}, \mathbf{w})$:

- $\mathbf{x}$ is the *explicit instance* of the relation.
- $\mathbf{y}$ is the *implicit instance* of the relation.
- $\mathbf{w}$ is the *witness* of the relation.

An *Interactive Oracle Proof of Proximity* (IOPP) [BCS16; RRR16] for a relation $\mathcal{R}$ is a tuple $\text{IOP} = (\mathbf{P}, \mathbf{V})$ that works as follows. The prover $\mathbf{P}$ is an interactive algorithm that receives as input $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$, and the verifier $\mathbf{V}$ is an efficient interactive oracle algorithm that receives input $\mathbf{x}$ and query access to $\mathbf{y}$ (consisting of l_0 symbols over some alphabet Σ_0). They interact over k rounds: in each round $i \in [k]$, $\mathbf{P}$ sends a proof string π_i consisting of l_i symbols over some alphabet Σ_i , and $\mathbf{V}$ sends a uniformly random message $\rho_i \in \{0, 1\}^{r_i}$. After k rounds, $\mathbf{V}$ queries symbols of $\mathbf{y}, \pi_1, \dots, \pi_k$ and accepts or rejects. We write $\langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})$ for this interaction.

Definition 2.5. A public-coin protocol has **randomness degree** $\theta_{\mathbf{V}} \geq 1$ if, on instances of length n , it has $k = O(n^{\theta_{\mathbf{V}}})$ rounds and $\sum_{j \in [k]} r_j = O(n^{\theta_{\mathbf{V}}})$.

Any IOP has a constant randomness degree (since the verifier is required to run in polynomial time).

Definition 2.6. $\text{IOP} = (\mathbf{P}, \mathbf{V})$ has **(perfect) completeness** if, for every $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \mathcal{R}$,

$$\Pr[\mathbf{V} \text{ accepts in } \langle \mathbf{P}, \mathbf{V} \rangle(\mathbf{x}, \mathbf{y}, \mathbf{w})] = 1.$$

Knowledge soundness for IOPs. Applications of IOPs typically require a strengthening of the classical notion of soundness, known as round-by-round knowledge soundness [BCFW25], which is needed for the Fiat–Shamir step [FS86] in the BCS [BCS16] transformation.

Definition 2.7. Let $\text{IOP} = (\mathbf{P}, \mathbf{V})$ be an IOP for a relation $\mathcal{R}$. A **knowledge state function for $\tilde{\mathcal{R}}$** is a (possibly inefficient) function State that, given a statement $(\mathbf{x}, \mathbf{y})$, an interaction transcript tr , and a **knowledge state witness** $\mathbf{w}$, outputs a bit and has the following properties.

- *Empty transcript:* if $\text{tr} = \emptyset$ is the empty transcript, then $\text{State}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $(\mathbf{x}, \mathbf{y}, \mathbf{w}) \in \tilde{\mathcal{R}}$.
- *Prover moves:* if tr is a transcript where the prover is about to move and $\text{State}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 0$, then $\text{State}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \pi, \mathbf{w}) = 0$ for every prover message π .
- *Full transcript:* if $\text{tr} = (\pi_1, \rho_1, \dots, \pi_k, \rho_k)$ is a full transcript, then $\text{State}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{w}) = 1$ if and only if $\mathbf{V}^{\mathbf{y}, \pi_1, \dots, \pi_k}(\mathbf{x}, \rho_1, \dots, \rho_k)$ accepts.

Definition 2.8. $\text{IOP} = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ has **round-by-round knowledge soundness for $\tilde{\mathcal{R}}$, errors $(\varepsilon_1, \dots, \varepsilon_k)$, and extraction times $(\text{et}_1, \dots, \text{et}_k)$** if there exist a knowledge state function State for $\tilde{\mathcal{R}}$ and a deterministic extractor $\mathbf{E}_{\text{rbr}}$ with the following property: for every statement $(\mathbf{x}, \mathbf{y})$, round index $i \in [k]$, and interaction transcript $\text{tr} = (\pi_1, \rho_1, \dots, \pi_{i-1}, \rho_{i-1}, \pi_i)$ where the verifier is about to move, $\mathbf{E}_{\text{rbr}}$ runs in time at most et_i and

$$\Pr_{\rho_i} \left[\exists \mathbf{w} : \begin{array}{l} \text{State}(\mathbf{x}, \mathbf{y}, \text{tr}, \mathbf{E}_{\text{rbr}}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w})) = 0 \\ \wedge \text{State}(\mathbf{x}, \mathbf{y}, \text{tr} \parallel \rho_i, \mathbf{w}) = 1 \end{array} \right] \leq \varepsilon_i(\mathbf{x}, \mathbf{y}).$$

The **total extraction time** is $\sum_{i \in [k]} \text{et}_i$.

2.3 Arguments in the random oracle model

We recall the definition of a non-interactive argument in the random oracle model (following [CY24]). For $\lambda \in \mathbb{N}$, let

$$\mathcal{H}_\lambda := \{h: \{0, 1\}^* \rightarrow \{0, 1\}^\lambda\}$$

be endowed with the uniform distribution; equivalently, the values $h(z)$ are uniform and mutually independent over distinct inputs z .

Definition 2.9. A **non-interactive argument in the ROM** for $\mathcal{R}$ is a pair $\text{ARG} = (\mathbf{P}, \mathbf{V})$ of oracle algorithms. Given oracle access to $h \leftarrow \mathcal{H}_\lambda$, the prover outputs $\pi \leftarrow \mathbf{P}^h(\mathfrak{x}, \mathfrak{w})$, and the verifier outputs a decision bit $\mathbf{V}^h(\mathfrak{x}, \pi)$. The argument has **perfect completeness** if, for every $\lambda \in \mathbb{N}$ and $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$,

$$\Pr \left[\mathbf{V}^h(\mathfrak{x}, \pi) = 1 \mid h \leftarrow \mathcal{H}_\lambda, \pi \leftarrow \mathbf{P}^h(\mathfrak{x}, \mathfrak{w}) \right] = 1.$$

We recall the notion of (adaptive) soundness for a non-interactive argument in the ROM.

Definition 2.10. An argument $\text{ARG} = (\mathbf{P}, \mathbf{V})$ for $\mathcal{R}$ has **adaptive soundness error** β if, for every output size $\lambda \in \mathbb{N}$, instance-size bound $n \in \mathbb{N}$, query bound $\mathfrak{q} \in \mathbb{N}$, and (possibly unbounded) $\mathfrak{q}$ -query malicious prover $\tilde{\mathbf{P}}$,

$$\Pr \left[\begin{array}{l} |\mathfrak{x}| \leq n \wedge \mathfrak{x} \notin \mathcal{L}(\mathcal{R}) \\ \wedge \mathbf{V}^h(\mathfrak{x}, \pi) = 1 \end{array} \mid \begin{array}{l} h \leftarrow \mathcal{H}_\lambda \\ (\mathfrak{x}, \pi) \leftarrow \tilde{\mathbf{P}}^h \end{array} \right] \leq \beta(\lambda, n, \mathfrak{q}).$$

We also recall (adaptive straightline) knowledge soundness.

Definition 2.11. An argument $\text{ARG} = (\mathbf{P}, \mathbf{V})$ for $\mathcal{R}$ has **adaptive straight-line knowledge soundness error** κ **with extraction time** et if there exists a probabilistic extractor $\mathbf{E}$ such that, for every $\lambda, n, \mathfrak{q} \in \mathbb{N}$ and every deterministic $\mathfrak{q}$ -query malicious prover $\tilde{\mathbf{P}}$,

$$\Pr \left[\begin{array}{l} |\mathfrak{x}| \leq n \wedge (\mathfrak{x}, \mathfrak{w}) \notin \mathcal{R} \wedge b = 1 \end{array} \mid \begin{array}{l} h \leftarrow \mathcal{H}_\lambda \\ (\mathfrak{x}, \pi; \tau_{\mathbf{P}}) \leftarrow \tilde{\mathbf{P}}^h \\ b \leftarrow \mathbf{V}^h(\mathfrak{x}, \pi) \\ \mathfrak{w} \leftarrow \mathbf{E}(\mathfrak{x}, \pi, \tau_{\mathbf{P}}) \end{array} \right] \leq \kappa(\lambda, n, \mathfrak{q}),$$

where $\tau_{\mathbf{P}}$ is the random-oracle query-answer trace of $\tilde{\mathbf{P}}$ and $\mathbf{E}$ runs in time $\text{et}(\lambda, n, \mathfrak{q})$.

2.4 The Fiat–Shamir transformation

A **public-coin interactive protocol** $\text{IP} = (\mathbf{P}, \mathbf{V})$ for a relation $\mathcal{R}$ works as follows. The prover receives $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$ and the verifier receives $\mathfrak{x}$. In each round $i \in [k]$, $\mathbf{P}$ sends a message m_i and $\mathbf{V}$ replies with a uniformly random $\rho_i \in \{0, 1\}^{r_i}$. After k rounds, $\mathbf{V}$ reads the transcript and accepts or rejects. This is the special case of Section 2.2 in which there is no implicit instance and the verifier reads every prover message in full; Definitions 2.5, 2.7 and 2.8 apply verbatim.

The Fiat–Shamir transformation compiles such a protocol into a non-interactive argument by deriving each verifier message from a hash of the transcript so far [FS86].

Construction 2.12 (Fiat–Shamir transformation). Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a k -round public-coin interactive protocol. Fix a round-domain-separated hash family h with output length λ . For each round i , fix a $\text{poly}(\lambda)$ -time map from $\{0, 1\}^\lambda$ to the verifier’s finite message space whose output on a uniform input is uniform. We denote the Fiat–Shamir transform by $\text{FS}^h(\text{IP})$.

- **Prover.** In round i , the prover computes m_i and the verifier message

$$\rho_i := \text{Challenge}_{h,i}(\text{id}(\mathfrak{x}), m_1, \dots, m_i) . \quad (2)$$

Here $\text{id}(\mathfrak{x})$ is the canonical statement encoding. The function $\text{Challenge}_{h,j}$ applies the round domain separator and canonical serialization, evaluates h , and applies the fixed map for round j . The prover outputs $\pi := (m_1, \dots, m_k)$.

- **Verifier.** The Fiat–Shamir verifier recomputes the verifier messages from π by (2) and accepts exactly when $\mathbf{V}$ accepts the resulting transcript.

The statement $\mathfrak{x}$ enters the first challenge. Without it the transform is only non-adaptively sound.

Theorem 2.13. *Let IP be as in Construction 2.12 and suppose that it has round-by-round knowledge soundness for $\tilde{\mathcal{R}}$, with errors $(\varepsilon_1, \dots, \varepsilon_k)$ and polynomial extraction time. Then $\text{FS}^h(\text{IP})$ has adaptive straight-line knowledge soundness for $\tilde{\mathcal{R}}$ in the random oracle model.*

More precisely, for every malicious argument prover making at most q random-oracle queries, there is a straight-line extractor whose knowledge error on $\mathfrak{x}$ is at most

$$(q + k) \cdot \max_{i \in [k]} \varepsilon_i(\mathfrak{x}) .$$

Proof. Round-by-round knowledge soundness gives straight-line state restoration with error at most $(q + k) \max_i \varepsilon_i$. Since the verifier reads the full transcript, the state-restoration extractor applied to the query–answer trace of the malicious prover gives adaptive straight-line knowledge soundness [BCS16; CY24]. $\square$

2.5 The BCS transformation

The Ben-Sasson–Chiesa–Spooner (BCS) transformation compiles a public-coin IOP into a non-interactive argument by Merkle-committing to each prover oracle and applying the Fiat–Shamir challenge rule to the Merkle roots [BCS16]; we follow the treatment in [CY24].

Construction 2.14 (BCS transformation). Let $\text{IOP} = (\mathbf{P}, \mathbf{V})$ be a k -round public-coin IOP without an implicit instance. Fix round-domain-separated hash families $h = (h_{\text{MT}}, h_{\text{FS}})$ with output length λ . For each round i , fix a $\text{poly}(\lambda)$ -time map from $\{0, 1\}^\lambda$ to the verifier’s finite message space whose output on a uniform input is uniform. We refer the reader to [CO25] for a systematic description. We denote the BCS transform by $\text{BCS}^h(\text{IOP})$.

- **Prover.** In round i , the prover computes π_i , its Merkle root

$$a_i := \text{MerkleRoot}_{h_{\text{MT}},i}(\pi_i) ,$$

and the verifier message

$$\rho_i := \text{Challenge}_{h_{\text{FS}},i}(\text{id}(\mathfrak{x}), a_1, \dots, a_i) . \quad (3)$$

Here $\text{id}(\mathbf{x})$ is the canonical statement encoding. The function $\text{Challenge}_{h,j}$ applies the round domain separator and canonical serialization, evaluates h , and applies the fixed map for round j .

After all rounds, the prover emulates $\mathbf{V}$ to obtain its query sets Q_i , and outputs

$$\pi := ((a_i, Q_i, \pi_i[Q_i], \omega_i))_{i \in [k]},$$

where ω_i contains the corresponding Merkle authentication paths.

- **Verifier.** The BCS verifier recomputes the verifier messages, emulates $\mathbf{V}$ on the authenticated entries, and accepts exactly when the IOP verifier and every Merkle opening accept.

We write $\varepsilon_{\text{BCS}}(\lambda, \mathbf{q}, (l_i)_{i \in [k]})$ for the explicit Merkle multi-extraction and Fiat-Shamir error in the BCS theorem of [CY24].

Theorem 2.15. *Let IOP be as in Construction 2.14 and suppose that it has round-by-round knowledge soundness for $\tilde{\mathcal{R}}$, with errors $(\varepsilon_1, \dots, \varepsilon_k)$ and polynomial extraction time. Then $\text{BCS}^h(\text{IOP})$ has adaptive straight-line knowledge soundness for $\tilde{\mathcal{R}}$ in the random oracle model.*

More precisely, for every malicious argument prover making at most $\mathbf{q}$ random-oracle queries, there is a straight-line extractor whose knowledge error on $\mathbf{x}$ is at most

$$(\mathbf{q} + k) \cdot \max_{i \in [k]} \varepsilon_i(\mathbf{x}, \perp) + \varepsilon_{\text{BCS}}(\lambda, \mathbf{q}, (l_i)_{i \in [k]}),$$

where ε_{BCS} is the bound defined above.

Proof. Round-by-round knowledge soundness gives straight-line state restoration with error at most $(\mathbf{q} + k) \max_i \varepsilon_i$. The BCS theorem then gives adaptive straight-line knowledge soundness with the additive error stated above [CY24]. $\square$

3 Diagonalization for programmable protocols

This section gives a general form of the diagonalization attack. We first define semantic schemes: relations whose membership is determined by a semantic value that is separate from the instance itself. Then, we define diagonalizable protocols, which we show to be susceptible to the techniques in this work.

3.1 Semantic schemes

A semantic scheme gives every instance $\mathfrak{x}$ a possible “meaning” σ , where that meaning contains exactly the semantic information needed to decide whether $\mathfrak{x}$ is in the relation.

Definition 3.1. *Let $\mathcal{R}$ be a relation over explicit instances $\mathfrak{x} \in \mathcal{X}$ and witnesses $\mathfrak{w}$. A **semantic scheme for $\mathcal{R}$** is a triple $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$, where Σ is a set of **semantic values**, $\Vdash \subseteq \mathcal{X} \times \Sigma$ is a **realization relation** and $\overline{\mathcal{R}}$ is a relation, such that $\mathfrak{x} \Vdash \sigma$ implies, for every $\mathfrak{w}$,*

$$(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R} \iff (\mathfrak{x}, \sigma, \mathfrak{w}) \in \overline{\mathcal{R}}.$$

Write $\mathcal{L}(\overline{\mathcal{R}}) = \{(\mathfrak{x}, \sigma) : \exists \mathfrak{w}, (\mathfrak{x}, \sigma, \mathfrak{w}) \in \overline{\mathcal{R}}\}$. If $\mathfrak{x} \Vdash \sigma$, then

$$\mathfrak{x} \in \mathcal{L}(\mathcal{R}) \iff (\mathfrak{x}, \sigma) \in \mathcal{L}(\overline{\mathcal{R}}).$$

While the definition of semantic schemes might seem vacuous (every relation has a semantic scheme, namely the one where $\overline{\mathcal{R}} = \mathcal{R}$ and $\Vdash$ is equality), this separation between instance and the value of the relation allows us to define schemes that are self-referential: the semantic value σ can depend (in an arbitrary way) on the instance $\mathfrak{x}$.

Definition 3.2. *Let $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$ be a semantic scheme for $\mathcal{R}$ and $\mathcal{F}$ a set of efficiently computable functions $\mathcal{X} \rightarrow \Sigma$. Then $\mathcal{S}$ is **self-referential for $\mathcal{F}$** if there is an algorithm Fix that, given the description of any $F \in \mathcal{F}$, runs in time $\text{poly}(|F|)$ and outputs an instance $\mathfrak{x}^* := \text{Fix}(F) \in \mathcal{X}$ satisfying*

$$\mathfrak{x}^* \Vdash F(\mathfrak{x}^*).$$

Self-referential schemes will be crucial for our attacks. Clearly, the trivial semantic scheme is self-referential for the function class $\mathcal{F}$ containing the identity function, but we will require a more expressive function class (in particular, F that can compute the Fiat–Shamir hash of $\mathfrak{x}$).

3.2 Programmability

We define the properties that (fixing a semantic scheme) make an interactive protocol susceptible to the results in this paper: programmability.

In a nutshell: a programmable prover sends its first i messages, then uses the verifier challenge prefix to *choose* a semantic value and (only then) completes the transcript. On every instance that realizes the chosen semantic value, the instance must be false and the verifier must accept (thus, breaking soundness).

Definition 3.3. *Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a k -round public-coin protocol for $\mathcal{R}$, let $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$ be a semantic scheme for $\mathcal{R}$, and fix $i \in \{0, 1, \dots, k\}$ and $\theta \geq 1$. The protocol is (i, θ) -**programmable***

relative to $\mathcal{S}$ if there are deterministic algorithms $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k, \text{Bind}$ such that $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_i, \text{Bind} \in \mathcal{F}_\theta$, the remaining provers run in polynomial time, and the following holds.

For every $\mathfrak{x} \in \mathcal{X}$ and every challenge sequence $\rho_1, \dots, \rho_k$, define

$$\begin{aligned} m_j &:= \tilde{\mathbf{P}}_j(\mathfrak{x}, \rho_{<j}) & (j \leq i), \\ \sigma &:= \text{Bind}(\mathfrak{x}, \rho_{\leq i}), \\ m_j &:= \tilde{\mathbf{P}}_j(\mathfrak{x}, \sigma, \rho_{<j}) & (j > i). \end{aligned}$$

If $\mathfrak{x} \models \sigma$, then

$$(\mathfrak{x}, \sigma) \notin \mathcal{L}(\overline{\mathcal{R}}) \quad \text{and} \quad \mathbf{V}(\mathfrak{x}, (m_j, \rho_j)_{j \in [k]}) = 1.$$

An interactive protocol that is programmable and whose semantic scheme is self-referential is susceptible⁴ to the diagonalization attack described in Section 3.3.

Definition 3.4. A public-coin protocol IP for $\mathcal{R}$ is (i, θ, θ') -**diagonalizable** if $\mathcal{R}$ has a semantic scheme $\mathcal{S}$ that is self-referential for $\mathcal{F}_\theta$ and IP is (i, θ') -programmable relative to $\mathcal{S}$.

Here θ bounds the complexity of the functions covered by self-reference, whereas θ' bounds that of the programmable provers and Bind in programmability.

3.3 Diagonalization attacks on programmable protocols

We first loosely bound the cost of computing the initial messages of a Fiat–Shamir compiled protocol when the hash function is polynomial-time computable.

Lemma 3.5. *Let:*

- IP be a public-coin interactive protocol with randomness degree $\theta_{\mathbf{V}}$;
- $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_i \in \mathcal{F}_\theta$; and
- h be a hash family evaluable in time $O(L^{\theta_h})$ on inputs of length L .

Then the maps

$$\mathfrak{x} \longmapsto (m_{\leq i}^h(\mathfrak{x}), \rho_{\leq i}^h(\mathfrak{x})),$$

defined by running $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_i$ against the Fiat–Shamir challenge rule (2) of Construction 2.12, are computable in time $O(i^{1+\theta_h} \cdot n^{\theta_{\mathbf{V}} \cdot \theta \cdot \theta_h}) \cdot \text{poly}(\lambda, |h|)$ given the code of h .

Proof. By induction on $j \leq i$. The message $m_j = \tilde{\mathbf{P}}_j(\mathfrak{x}, \rho_{<j})$ depends only on data already computed. Randomness degree $\theta_{\mathbf{V}}$ gives $\sum_{j \leq k} r_j = O(n^{\theta_{\mathbf{V}}})$; hence $\tilde{\mathbf{P}}_j$ runs on an input of length $O(n^{\theta_{\mathbf{V}}})$ and both its cost and its output bit length are $O(n^{\theta_{\mathbf{V}} \theta})$. By (2), ρ_j is one evaluation of $\text{Challenge}_{h,j}$, on $(\text{id}(\mathfrak{x}), m_1, \dots, m_j)$, of length $O(j n^{\theta_{\mathbf{V}} \theta} + \lambda)$, costing $O(j^{\theta_h} n^{\theta_{\mathbf{V}} \theta \theta_h}) \cdot \text{poly}(\lambda, |h|)$. Summing over the i rounds gives the claim. $\square$

Remark 3.6. The above lemma is intentionally loose. For specific protocols and hash functions the complexity to compute the Fiat–Shamir hashes can be much smaller (in fact, often linear in the prover message length).

We show that diagonalizable interactive protocols are unsound when compiled via Fiat–Shamir for any sufficiently efficiently computable hash function h .

⁴We remark that the condition is sufficient but not necessary.

Theorem 3.7. *Let IP be an (i, θ, θ') -diagonalizable public-coin protocol for $\mathcal{R}$, for some $i \in [k]$, and let its randomness degree be $\theta_{\mathbf{V}}$. For every hash family h evaluable in time $O(L^{\theta_h})$ such that*

$$\theta_{\mathbf{V}} (1 + (\theta' + 1)\theta_h) \leq \theta,$$

one can efficiently construct $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R})$ and a proof π_h accepted by $\text{FS}^h(\text{IP})$.

Proof. Fix h . Let $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k$, Bind witness programmability, and let Fix witness self-reference. Run the first i provers with Fiat–Shamir challenges as in Lemma 3.5, and define

$$F_h(\mathbf{x}) := \text{Bind}(\mathbf{x}, \rho_{\leq i}^h(\mathbf{x})). \quad (4)$$

By Lemma 3.5, F_h is computable in time

$$O\left(i^{1+\theta_h} n^{\theta_{\mathbf{V}}\theta'\theta_h}\right) \cdot \text{poly}(\lambda, |h|).$$

Moreover, $i \leq k = O(n^{\theta_{\mathbf{V}}})$. Since h is fixed before F_h is formed, the exponent assumption gives $F_h \in \mathcal{F}_{\theta}$, and its code is efficiently obtained from that of h .

Set $\mathbf{x}_h := \text{Fix}(F_h)$. By self-reference,

$$\mathbf{x}_h \models F_h(\mathbf{x}_h) = \text{Bind}(\mathbf{x}_h, \rho_{\leq i}^h(\mathbf{x}_h)) =: \sigma_h.$$

The malicious prover now repeats the first i messages and challenges used to define F_h , then runs the remaining provers on $(\mathbf{x}_h, \sigma_h)$, deriving every later challenge by the Fiat–Shamir rule. Let π_h be the resulting message sequence.

By Definition 3.3 $(\mathbf{x}_h, \sigma_h) \notin \mathcal{L}(\overline{\mathcal{R}})$ and the interactive verifier accepts the transcript. By the semantic-scheme property, $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R})$. The Fiat–Shamir verifier recomputes the same challenges, so it must accept π_h . All steps are polynomial time by self-reference, Lemma 3.5, and the running-time requirements in Definition 3.3. $\square$

3.4 Lifting the attack to IOPs

Since the [BCS16] transformation can be conceptualized as the Fiat–Shamir transformation applied to the interactive argument where prover messages are replaced with Merkle commitments, the attack translates directly with minor adaptations.

Definition 3.8. *The underlying interactive protocol of a public-coin IOP $\text{IOP} = (\mathbf{P}, \mathbf{V})$ is the public-coin interactive protocol whose prover messages are the strings $\pi_1, \dots, \pi_k$ and whose decision is $\mathbf{V}^{\pi_1, \dots, \pi_k}(\mathbf{x}, \rho_1, \dots, \rho_k)$. We call IOP **programmable**, respectively **diagonalizable**, with given parameters if its underlying interactive protocol is.*

Lemma 3.9. *Lemma 3.5 holds with running time $O(i n^{\theta_{\mathbf{V}}\theta\theta_h}) \cdot \text{poly}(\lambda, |h|)$ for a public-coin IOP IOP and a pair of hash families $h = (h_{\text{MT}}, h_{\text{FS}})$, with the maps defined by the BCS challenge rule (3) of Construction 2.14.*

Proof. As in Lemma 3.5, the message $\pi_j = \tilde{\mathbf{P}}_j(\mathbf{x}, \rho_{\leq j})$ has cost and bit length $B = O(n^{\theta_{\mathbf{V}}\theta})$. Computing the Merkle root $a_j = \text{MerkleRoot}_{h_{\text{MT}}, j}(\pi_j)$ costs $O(n^{\theta_{\mathbf{V}}\theta\theta_h}) \cdot \text{poly}(\lambda, |h|)$: the leaves partition the B bits, so hashing all of them costs $\sum_k O(\ell_k^{\theta_h}) \leq O(B^{\theta_h})$ as $\theta_h \geq 1$, while the $O(B)$ internal nodes hash $O(\lambda)$ bits each. The challenge ρ_j is one evaluation of $\text{Challenge}_{h_{\text{FS}}, j}$ on an input of length $O(n + j\lambda)$, which the Merkle term dominates up to $\text{poly}(\lambda)$ since $j = O(n^{\theta_{\mathbf{V}}})$. Summing over the i rounds gives the claim. $\square$

Theorem 3.10. *Let IOP be an (i, θ, θ') -diagonalizable public-coin IOP for $\mathcal{R}$, for some $i \in [k]$, and let its randomness degree be $\theta_{\mathbf{V}}$. For every pair of hash families $h = (h_{\text{MT}}, h_{\text{FS}})$ evaluable in time $O(L^{\theta_h})$ such that*

$$\theta_{\mathbf{V}}(1 + \theta'\theta_h) \leq \theta,$$

one can efficiently construct $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R})$ and a proof π_h accepted by $\text{BCS}^h(\text{IOP})$.

Proof. Repeat the proof of Theorem 3.7, using the BCS challenge rule and Lemma 3.9. This gives a fixed-point instance $\mathbf{x}_h$ that realizes σ_h . Programmability and the semantic-scheme property make the instance false and give oracle messages $\pi_1, \dots, \pi_k$ accepted by the IOP verifier.

To form the BCS proof, emulate the verifier to obtain its query sets $Q_1, \dots, Q_k$ and output

$$\pi_h := ((a_j, Q_j, \pi_j[Q_j], \omega_j))_{j \in [k]},$$

where each a_j is the genuine Merkle root and ω_j contains the genuine authentication paths. The BCS verifier derives the same challenges, all Merkle openings verify, and the underlying IOP verifier accepts. $\square$

The exponent hypothesis above is loose, as it composes the cost of Bind with that of h over all k rounds. In our attacks the first malicious message is a fixed string, and the following sharpening applies.

Corollary 3.11. *Let IOP be a $(1, \theta, \theta')$ -diagonalizable public-coin IOP for $\mathcal{R}$ whose first malicious prover $\hat{\mathbf{P}}_1$ outputs a fixed string π_1° , independent of the instance. For every pair of hash families $h = (h_{\text{MT}}, h_{\text{FS}})$ evaluable in time $O(L^{\theta_h})$ such that*

$$\max\{\theta', \theta_h\} \leq \theta,$$

one can efficiently construct $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R})$ and a proof π_h accepted by $\text{BCS}^h(\text{IOP})$. Moreover $\mathbf{x}_h = \text{Fix}(F_h)$ for the diagonal map

$$F_h(\mathbf{x}) := \text{Bind}(\mathbf{x}, \text{Challenge}_{h_{\text{FS}}, 1}(\text{id}(\mathbf{x}), a_1^\circ)), \quad a_1^\circ := \text{MerkleRoot}_{h_{\text{MT}}, 1}(\pi_1^\circ). \quad (5)$$

Proof. Repeat the proof of Theorem 3.10. Its only use of the exponent hypothesis is to place the diagonal map (4) in $\mathcal{F}_\theta$, and for $i = 1$ that map is (5). Since π_1° is constant, so is the root a_1° , which is computed once and hard-coded. On an instance of length N , F_h therefore writes $\text{id}(\mathbf{x})$ in $O(N)$ steps, calls h_{FS} once at cost $O(N^{\theta_h})$, and runs $\text{Bind} \in \mathcal{F}_{\theta'}$; the factor $\text{poly}(\lambda, |h|)$ is absorbed by padding, as in Lemma 3.5. Hence $F_h \in \mathcal{F}_{\max\{\theta', \theta_h\}} \subseteq \mathcal{F}_\theta$. $\square$

4 Generated relations

In this section we show that *every* relation admits a **generated** variant that has a self-referential semantic scheme (Definition 3.2). We then instantiate the construction to obtain generated R1CS, the relation we consider in Sections 6 and 7.

4.1 Universal computation

Let ϵ be the empty string. Fix a standard programming system with a universal evaluator $\mathfrak{U}$ and efficient program specialization.⁵ For a program P , let $\text{code}(P)$ denote its code. For a code s , let $\mathfrak{R}_s(x) := \mathfrak{U}(s, x)$ be the partial function computed by s . We write $\mathfrak{R}_s(x) \downarrow_{\leq T} = y$ if it halts within T steps and outputs y . Specialization has linear code-size overhead. The evaluator simulates t steps of a code s in time $t \cdot \text{poly}(|s|, \log t)$. Standard RAM-based programming systems have these properties.

We use the following constructive form of Kleene's second recursion theorem [Kle52].

Lemma 4.1. *For every program F , one can efficiently construct a code s^* such that*

$$\mathfrak{R}_{s^*}(\epsilon) = F(s^*).$$

The construction has linear code-size overhead. Executing s^ adds only the polynomial simulation overhead stated above.*

Proof. For each code a , let

$$\text{diag}(a) := \text{code}(\epsilon \mapsto \mathfrak{U}(a, a)).$$

The map diag is efficiently computable. Let

$$b := \text{code}(a \mapsto F(\text{diag}(a))) \quad \text{and} \quad s^* := \text{diag}(b).$$

Then

$$\mathfrak{R}_{s^*}(\epsilon) = \mathfrak{U}(\text{diag}(b), \epsilon) = \mathfrak{U}(b, b) = F(\text{diag}(b)) = F(s^*).$$

□

4.2 Generated relations

Given any relation, we define a variant in which the instance is not written down explicitly, but is instead generated by a program. The variant is parameterized by a constant $c > 1$, the **budget exponent**: an instance with clock T permits its generator to run for T^c steps.

Definition 4.2. *Let $\mathcal{R}$ be a relation and let $c \geq 2$ be an integer. The **c-generated relation** $\mathcal{R}[c]$ consists of tuples $(\mathfrak{x}, \mathfrak{w})$ of the form $\mathfrak{x} = (1^T, s)$, where $T \in \mathbb{N}$ and $s \in \{0, 1\}^*$, such that*

$$\mathfrak{R}_s(\epsilon) \downarrow_{\leq T^c} = \sigma \quad \text{with} \quad |\sigma| \leq T \quad \text{and} \quad (\sigma, \mathfrak{w}) \in \mathcal{R}.$$

⁵Given code for P and fixed data a , specialization produces code for $x \mapsto P(a, x)$ without running P . Program code may also be used as data. These are the standard acceptability conditions.

If $\mathcal{R} \in \text{NP}$ then $\mathcal{R}[\mathfrak{c}] \in \text{NP}$: the unary clock lets the verifier simulate the generator for $T^{\mathfrak{c}}$ steps, and the size bound $|\sigma| \leq T$ keeps witnesses of polynomial size. Moreover $\mathcal{L}(\mathcal{R})$ reduces to $\mathcal{L}(\mathcal{R}[\mathfrak{c}])$ by mapping σ to $(1^T, s)$ where s simply prints σ and $T = O(|\sigma|)$. In particular, $\mathcal{L}(\mathcal{R}[\mathfrak{c}])$ is NP-complete whenever $\mathcal{L}(\mathcal{R})$ is.

We next prove a lemma that shows how to construct (efficiently) self-referential instances for generated relations.

Lemma 4.3. *For a clock T and code s , write $\mathfrak{x}_{T,s} := (1^T, s)$. Fix an integer $\mathfrak{c} \geq 2$, a degree $\theta < \mathfrak{c}$, and a bound $B \in \mathbb{N}$. Let F be a θ -tame program that, on input $\mathfrak{x}_{T,s}$, outputs a string of length at most B . Then one can efficiently construct T^* and s^* such that*

$$\mathfrak{R}_{s^*}(\epsilon) \downarrow_{\leq (T^*)^{\mathfrak{c}}} = F(\mathfrak{x}_{T^*, s^*}).$$

Moreover, $T^* \geq B$ (so the output has length at most T^*) and $T^* = \text{poly}(|F|, B)$.

Proof. For each T , let F_T be the code that, on input a , constructs $\mathfrak{x}_{T,a}$ and runs F on it. The code stores T in binary and expands its unary encoding only when run. Thus $|F_T| = O(|F| + \log T)$. Constructing the input takes $O(T)$ time, which is covered because $\theta \geq 1$.

Apply Lemma 4.1 to F_T . This gives, efficiently in T , a code s_T such that

$$\mathfrak{R}_{s_T}(\epsilon) = F_T(s_T) = F(\mathfrak{x}_{T, s_T}).$$

The recursion construction has linear code-size overhead. Hence $|s_T| = O(|F| + \log T)$ and

$$n_T := |\mathfrak{x}_{T, s_T}| = O(D + T), \text{ where } D := |F|.$$

By Definition 2.1, F runs in at most $(|F| + 1)(n_T + 1)^\theta$ steps. The fixed recursion construction and the universal evaluator add a factor polynomial in $|s_T|$ and the logarithm of this running time. Therefore, for constants c, q that do not depend on F or T ,

$$\text{time}(\mathfrak{R}_{s_T}(\epsilon)) \leq c(D + T)^\theta (D + \log T + 1)^q.$$

Choose a constant θ' with $\theta < \theta' < \mathfrak{c}$. For all $T \geq D^r$, where r is a sufficiently large constant, the last display is at most $c'T^{\theta'}$. Thus it is at most $T^{\mathfrak{c}}$ for all $T \geq T_0$, for some $T_0 = \text{poly}(D)$.

To find a valid clock, try $T = 2, 4, 8, \dots$. For each value, construct s_T and simulate it for $T^{\mathfrak{c}}$ steps. Let T^* be the first value with $T^* \geq B$ for which the simulation halts. The search succeeds with $T^* \leq 2 \max(T_0, B) = \text{poly}(|F|, B)$ and takes $\text{poly}(T^*)$ total time. Setting $s^* := s_{T^*}$ proves the claim. $\square$

Using the lemma, we show *every* generated relation is self-referential.

Lemma 4.4. *Let $\mathcal{R}$ be a relation and fix integers $\mathfrak{c} \geq 2$ and $B \in \mathbb{N}$. There exists a semantic scheme $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$ for $\mathcal{R}[\mathfrak{c}]$, whose semantic values Σ are all strings of length at most B and whose realization relation is*

$$(1^T, s) \Vdash \sigma \iff \mathfrak{R}_s(\epsilon) \downarrow_{\leq T^{\mathfrak{c}}} = \sigma \text{ and } |\sigma| \leq T,$$

that is self-referential for $\mathcal{F}_\theta$, for every $\theta < \mathfrak{c}$, with $|\text{Fix}(F)| = \text{poly}(|F|, B)$.

Proof. Put $\Sigma := \{\sigma \in \{0, 1\}^* \text{ s.t. } |\sigma| \leq B\}$ and let $\Vdash$ be as in the statement; note that Σ contains every candidate instance of length at most B , not only those in $\mathcal{L}(\mathcal{R})$. Let $\overline{\mathcal{R}}$ consist of the triples $(\mathfrak{x}, \sigma, \mathfrak{w})$ with $(\sigma, \mathfrak{w}) \in \mathcal{R}$. Set $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$. For the semantic-scheme property, suppose $\mathfrak{x} \Vdash \sigma$. Since the generator is deterministic, σ is the unique instance it produces within the budget, so by Definition 4.2, for every $\mathfrak{w}$, $(\mathfrak{x}, \mathfrak{w}) \in \mathcal{R}[c]$ if and only if $(\sigma, \mathfrak{w}) \in \mathcal{R}$, i.e. if and only if $(\mathfrak{x}, \sigma, \mathfrak{w}) \in \overline{\mathcal{R}}$.

For self-reference, fix $\theta < c$ and let F lie in $\mathcal{F}_\theta$. Its outputs have length at most B , so Lemma 4.3 yields $T^* \geq B$ and s^* with

$$\mathfrak{R}_{s^*}(\epsilon) \downarrow_{\leq (T^*)^c} = F(\mathfrak{x}_{T^*, s^*}).$$

Set

$$\text{Fix}(F) := (1^{T^*}, s^*).$$

This is efficient, and since $|F(\mathfrak{x}_{T^*, s^*})| \leq B \leq T^*$ we get $\text{Fix}(F) \Vdash F(\text{Fix}(F))$. The bound on T^* also gives $|\mathfrak{x}^*| = \text{poly}(|F|, B)$. $\square$

4.3 Generated R1CS

We instantiate the construction with the rank-1 constraint satisfaction (R1CS) relation, which yields the relation attacked in Sections 6 and 7.

Definition 4.5. *The R1CS relation $\mathcal{R}_{\text{R1CS}}$ consists of tuples $(\mathfrak{x}, \mathfrak{w})$ where $\mathfrak{x} = (\mathbb{F}, M, N, \mathbf{A}, \mathbf{B}, \mathbf{C})$ for a finite field $\mathbb{F}$, dimensions $M, N \geq 1$ and matrices $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}^{M \times N}$, and $\mathfrak{w} = \mathbf{z} \in \mathbb{F}^N$ is such that $z_0 = 1$ and*

$$(\mathbf{A}\mathbf{z}) \circ (\mathbf{B}\mathbf{z}) = \mathbf{C}\mathbf{z}.$$

We let $B(\mathbb{F}, M, N)$ denote the length of the encoding of $\mathfrak{x}$; note that $B(\mathbb{F}, M, N) = O(MN \log |\mathbb{F}|)$.

Definition 4.6. *For an integer $c \geq 2$, the generated R1CS relation is $\mathcal{R}_{\text{R1CS}}[c]$, the c -generated relation of $\mathcal{R}_{\text{R1CS}}$.*

Since $\mathcal{L}(\mathcal{R}_{\text{R1CS}})$ is NP-complete, so is $\mathcal{L}(\mathcal{R}_{\text{R1CS}}[c])$. Specializing Lemma 4.4 gives the semantic scheme used in Sections 6 and 7.

Corollary 4.7. *Fix an integer $c \geq 2$, a finite field $\mathbb{F}$, and dimensions $M, N \geq 1$, and put $B := B(\mathbb{F}, M, N)$. There is a semantic scheme $\mathcal{S} = (\Sigma, \Vdash, \overline{\mathcal{R}})$ for $\mathcal{R}_{\text{R1CS}}[c]$ that is self-referential for $\mathcal{F}_\theta$, for every $\theta < c$, with $|\text{Fix}(F)| = \text{poly}(|F|, M, N, \log |\mathbb{F}|)$.*

Proof. Apply Lemma 4.4 to $\mathcal{R}_{\text{R1CS}}$ with the bound B . Every R1CS instance over $\mathbb{F}$ with dimensions $M \times N$ has encoding length exactly B , hence lies in Σ . Finally $B = O(MN \log |\mathbb{F}|)$, so $\text{poly}(|F|, B) = \text{poly}(|F|, M, N, \log |\mathbb{F}|)$. $\square$

5 Lifting protocols to semantic relations

We formalize lifting a protocol from statements to encodings of statements.

Definition 5.1 (Semantic lift). *Let $\mathcal{R}$ be a relation over Σ and let $\text{val}: \mathcal{X} \rightarrow \Sigma$ be an efficient deterministic value map⁶. The lift of $\mathcal{R}$ along val is*

$$\mathcal{R}[\text{val}] := \{(\mathfrak{x}, \mathfrak{w}) : (\text{val}(\mathfrak{x}), \mathfrak{w}) \in \mathcal{R}\}. \quad (6)$$

The lift of a public-coin protocol $\text{IP} = (\mathbf{P}, \mathbf{V})$ for $\mathcal{R}$ is the protocol $\text{IP}[\text{val}]$ that, on input $\mathfrak{x}$, runs IP on $\text{val}(\mathfrak{x})$.

Completeness and soundness transfer immediately. The lifted relation $\mathcal{R}[\text{val}]$ has a **canonical semantic scheme**

$$\mathfrak{x} \Vdash \sigma \iff \text{val}(\mathfrak{x}) = \sigma, \quad (\mathfrak{x}, \sigma, \mathfrak{w}) \in \overline{\mathcal{R}} \iff (\sigma, \mathfrak{w}) \in \mathcal{R}.$$

In particular, the generated relations of Section 4 are lifted relations, where the value map is the evaluation map of Definition 4.2 (with invalid evaluations mapped to $\perp$).

Definition 5.2. *Let $\text{IP} = (\mathbf{P}, \mathbf{V})$ be a k -round public-coin protocol for $\mathcal{R}$. It has (i, θ') -delayed statement choice if there are algorithms $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_k, \text{Bind}$, where $\tilde{\mathbf{P}}_1, \dots, \tilde{\mathbf{P}}_i, \text{Bind} \in \mathcal{F}_{\theta'}$ and the remaining algorithms are polynomial-time, such that, for every string z and challenge sequence, setting*

$$\begin{aligned} m_j &= \tilde{\mathbf{P}}_j(z, \rho_{<j}) & (j \leq i), \\ \sigma &= \text{Bind}(z, \rho_{\leq i}), \\ m_j &= \tilde{\mathbf{P}}_j(z, \sigma, \rho_{<j}) & (j > i) \end{aligned}$$

gives

$$\sigma \notin \mathcal{L}(\mathcal{R}) \quad \text{and} \quad \mathbf{V}(\sigma, (m_j, \rho_j)_{j \in [k]}) = 1.$$

We show that if IP has delayed statement choice and the scheme is self-referential, the lifted protocol $\text{IP}[\text{val}]$ is susceptible to Theorem 3.7.

Proposition 5.3. *If IP permits (i, θ') -delayed statement choice and the canonical scheme is self-referential for $\mathcal{F}_\theta$, the lift $\text{IP}[\text{val}]$ is (i, θ, θ') -diagonalizable. Hence Theorem 3.7 applies.*

Proof. Run the delayed-choice algorithms with $z = \mathfrak{x}$. If $\mathfrak{x} \Vdash \sigma$, then $\text{val}(\mathfrak{x}) = \sigma$; thus the lifted verifier is $\mathbf{V}(\sigma, \cdot)$ and Definition 3.3 holds. By Definition 3.4, then the preconditions of Theorem 3.7 hold. $\square$

Fiat–Shamir compilation of lifted protocols. We show that the lifted protocols can be compiled by a slight modification to the Fiat–Shamir transformation, making the protocol as secure as the original protocol (after the standard Fiat–Shamir transformation).

Construction 5.4 (Semantic Fiat–Shamir). The transform $\text{FS}_{\text{sem}}^h(\text{IP}[\text{val}])$ is Construction 2.12 with challenges

$$\rho_i := \text{Challenge}_{h,i}(\text{id}(\text{val}(\mathfrak{x})), m_1, \dots, m_i). \quad (7)$$

⁶A partial value map can be made total by mapping invalid encodings to $\perp \notin \mathcal{L}(\mathcal{R})$.

Proposition 5.5. *For every concrete hash family h , the soundness (resp. knowledge soundness) error of $\text{FS}_{\text{sem}}^h(\text{IP}[\text{val}])$ is upper-bounded by that of $\text{FS}^h(\text{IP})$. The reduction incurs the cost of one evaluation of val .*

Proof. Fix (x, π) and put $\sigma := \text{val}(x)$. In the first round, the semantic verifier hashes $(\text{id}(\sigma), m_1)$ by (7), exactly as the base Fiat–Shamir verifier on statement σ . In every round j , both verifiers hash the same tuple $(\text{id}(\sigma), m_1, \dots, m_j)$, so all challenges agree. Both verifiers then run $\mathbf{V}$ on σ and the same transcript, so they make the same decision.

Let $\tilde{\mathbf{P}}$ be a standard-model adversary against the semantic transform. The base adversary runs $\tilde{\mathbf{P}}$ and maps its output (x, π) to $(\text{val}(x), \pi)$. By (6),

$$x \notin \mathcal{L}(\mathcal{R}[\text{val}]) \iff \text{val}(x) \notin \mathcal{L}(\mathcal{R}).$$

This identity, together with the agreement of decisions above, shows that the two adversaries succeed on exactly the same executions.

For knowledge soundness, apply the base extractor to the induced adversary on $(\text{val}(x), \pi)$. By (6), every witness it outputs is also a witness for x . Thus the knowledge error does not increase. In both reductions the only additional cost is one evaluation of val . $\square$

6 BCS attacks against multilinear zerocheck

In this section, we present a natural protocol for generated R1CS that resembles multilinear proof systems used in practice (e.g., [Set20; ACFY25; BRW26]). The protocol performs a multilinear zerocheck and then runs a lincheck.

6.1 Preliminaries

We recall the linear constraint relation.

Definition 6.1. Let $\mathcal{C}: \mathbb{F}^k \rightarrow \Sigma^\ell$ be an error-correcting code. The **linear constraint relation** $\mathcal{R}_\mathcal{C}$ consists of tuples $(\mathbf{x}, \mathbf{y}, \mathbf{w})$ of the form $\mathbf{x} = (\mu, \mathbf{v}) \in \mathbb{F} \times \mathbb{F}^k$, $\mathbf{y} = f \in \Sigma^\ell$ and $\mathbf{w} = \mathbf{z} \in \mathbb{F}^k$ such that $f = \mathcal{C}(\mathbf{z})$ and $\langle \mathbf{z}, \mathbf{v} \rangle = \mu$.

Many IOPPs and interactive linear oracle commitments use this relation [DP26; BFRW26]. Examples appear in [ZCF24; ACFY25] and [BCFRRZ25; NA25].

6.2 Construction

We describe the protocol we target next. Fix a budget exponent $c \geq 2$.

Construction 6.2. The public-coin IOP $\text{IOP}_{\text{ml}} = (\mathbf{P}, \mathbf{V})$ is defined as follows. Let $\mathcal{C}: \mathbb{F}^N \rightarrow \Sigma^\ell$ be an error-correcting code with polynomial block length and a polynomial-time encoder. Let $(\mathbf{P}_\mathcal{C}, \mathbf{V}_\mathcal{C})$ be an IOPP for $\mathcal{R}_\mathcal{C}$. The protocol is for the generated R1CS relation $\mathcal{R}_{\text{R1CS}}[\mathcal{C}]$, restricted to instances whose generated matrices are over $\mathbb{F}$ with dimensions $M = 2^m$ and $N = 2^n$.

- **Inputs.** The verifier $\mathbf{V}$ receives $\mathbf{x} = (1^T, s)$. The honest prover $\mathbf{P}$ additionally receives $\mathbf{w} = \mathbf{z}$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\text{R1CS}}[\mathcal{C}]$.
- **Interactive phase.**
 1. **Compute R1CS matrices.** The parties compute $\sigma := \mathfrak{R}_s(\epsilon)$. The verifier rejects unless the execution halts within T^c steps and σ is an R1CS instance $(\mathbb{F}, M, N, \mathbf{A}, \mathbf{B}, \mathbf{C})$ over $\mathbb{F}$ with the prescribed dimensions $M = 2^m$ and $N = 2^n$, of encoding length $|\sigma| \leq T$. Let $\hat{A}, \hat{B}, \hat{C}$ denote the multilinear extensions of the matrices, with rows indexed by $\{0, 1\}^m$ and columns by $\{0, 1\}^n$.
 2. **Witness oracle.** The prover sends its first oracle $\pi_1 = f := \mathcal{C}(\mathbf{z})$.
 3. **Zerocheck randomness.** The verifier samples and sends $\mathbf{r} \leftarrow \mathbb{F}^m$.
 4. **Zerocheck.** Define

$$\hat{G}(\mathbf{X}) := \text{eq}(\mathbf{X}, \mathbf{r}) \cdot \left(\left(\sum_{\mathbf{c} \in \{0, 1\}^n} \hat{A}(\mathbf{X}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}) \right) \cdot \left(\sum_{\mathbf{c} \in \{0, 1\}^n} \hat{B}(\mathbf{X}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}) \right) - \sum_{\mathbf{c} \in \{0, 1\}^n} \hat{C}(\mathbf{X}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}) \right).$$

The parties run the m -round sumcheck protocol on the claim

$$\sum_{\mathbf{b} \in \{0, 1\}^m} \hat{G}(\mathbf{b}) = 0.$$

Let $\boldsymbol{\gamma} \in \mathbb{F}^m$ be the terminal sumcheck point, and let $\hat{h}_m$ be the final prover message.

5. **Terminal evaluations.** The prover sends

$$\mu_A := \sum_{\mathbf{c} \in \{0,1\}^n} \hat{A}(\boldsymbol{\gamma}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}), \quad \mu_B = \sum_{\mathbf{c} \in \{0,1\}^n} \hat{B}(\boldsymbol{\gamma}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}), \quad \mu_C = \sum_{\mathbf{c} \in \{0,1\}^n} \hat{C}(\boldsymbol{\gamma}, \mathbf{c}) \cdot \mathbf{z}(\mathbf{c}).$$

The verifier checks $\hat{h}_m(\gamma_m) = \text{eq}(\boldsymbol{\gamma}, \mathbf{r}) \cdot (\mu_A \mu_B - \mu_C)$.

6. **Lincheck.** The verifier samples $\varepsilon \leftarrow \mathbb{F}$ and defines

$$\mathbf{v} := \mathbf{e}_0 + \varepsilon \cdot \hat{A}(\boldsymbol{\gamma}, \cdot) + \varepsilon^2 \cdot \hat{B}(\boldsymbol{\gamma}, \cdot) + \varepsilon^3 \cdot \hat{C}(\boldsymbol{\gamma}, \cdot).$$

The parties run $(\mathbf{P}_c, \mathbf{V}_c)$ to check that

$$(\mathbf{x} = (1 + \varepsilon \mu_A + \varepsilon^2 \mu_B + \varepsilon^3 \mu_C, \mathbf{v}), \mathbf{y} = f, \mathbf{w} = \mathbf{z}) \in \mathcal{R}_c.$$

- **Decision phase.** The verifier accepts if and only if the generator terminates correctly, every sumcheck and terminal check passes, and the lincheck proof verifies.

6.3 Soundness proof

For $\delta \in (0, 1)$, let $\mathcal{R}_{c,\delta}$ denote the δ -relaxation of $\mathcal{R}_c$, obtained by replacing the condition $f = \mathcal{C}(\mathbf{z})$ with $\Delta(f, \mathcal{C}(\mathbf{z})) \leq \delta$. Write k_c for the number of rounds of the lincheck, and let $(\varepsilon_{c,j}(\delta))_{j \in [k_c]}$ be its round-by-round knowledge errors for $\mathcal{R}_{c,\delta}$. The proof of the following theorem is essentially unchanged from prior work [ACFY25].

Theorem 6.3. *For every $\delta \in (0, 1)$ such that $(\mathbf{P}_c, \mathbf{V}_c)$ has round-by-round knowledge soundness for $\mathcal{R}_{c,\delta}$ with polynomial extraction time, the protocol IOP_{ml} has round-by-round knowledge soundness for $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$, with polynomial extraction time and round errors*

$$\left(\frac{m |\Lambda(\mathcal{C}, \delta)|}{|\mathbb{F}|}, \underbrace{\frac{3 |\Lambda(\mathcal{C}, \delta)|}{|\mathbb{F}|}, \dots, \frac{3 |\Lambda(\mathcal{C}, \delta)|}{|\mathbb{F}|}}_{m \text{ sumcheck rounds}}, \frac{3 |\Lambda(\mathcal{C}, \delta)|}{|\mathbb{F}|}, (\varepsilon_{c,j}(\delta))_{j \in [k_c]} \right).$$

In particular, the error in every round is at most

$$\varepsilon_{\text{ml}}(\delta) := \max \left\{ \frac{\max\{m, 3\} \cdot |\Lambda(\mathcal{C}, \delta)|}{|\mathbb{F}|}, \max_{j \in [k_c]} \varepsilon_{c,j}(\delta) \right\}. \quad (8)$$

Here $|\Lambda(\mathcal{C}, \delta)|$ is the maximum number of codewords in a ball of radius δ , as in Section 2.1.

Combining Theorem 6.3 with the BCS knowledge-soundness theorem gives the soundness of the compiled protocol.

Corollary 6.4 (Soundness of the compiled protocol). *Under the hypotheses of Theorem 6.3, let $k_{\text{ml}} = m + 2 + k_c$ be the number of rounds of IOP_{ml} . In the random oracle model, for every malicious argument prover making at most $\mathbf{q}$ random-oracle queries, there is an adaptive straight-line extractor for $\text{BCS}^h(\text{IOP}_{\text{ml}})$ whose knowledge error for $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$ is at most*

$$(\mathbf{q} + k_{\text{ml}}) \cdot \varepsilon_{\text{ml}}(\delta) + \varepsilon_{\text{BCS}} \left(\lambda, \mathbf{q}, (l_i)_{i \in [k_{\text{ml}}]} \right).$$

Proof. Apply Theorem 2.15 with the uniform round error from (8). □

6.4 Attack

The attack is an instance of Theorem 3.10. Write $\mathcal{S}$ for the semantic scheme of Corollary 4.7 for the field $\mathbb{F}$ and dimensions $2^m \times 2^n$, so that every R1CS instance over $\mathbb{F}$ with these dimensions is a semantic value; the first challenge $\mathbf{r}$ is the only one the attack needs to control, so $i = 1$.

Lemma 6.5. *Assume $m \geq 1$ and that the lincheck has an efficient prover and perfect completeness. Then IOP_{ml} is $(1, 1)$ -programmable relative to $\mathcal{S}$.*

Proof. Let $\tilde{\mathbf{P}}_1$ ignore its input and output $f^\circ := \mathcal{C}(\mathbf{z}^\circ)$ for $\mathbf{z}^\circ := \mathbf{e}_0$, and let

$$\text{Bind}(\mathbf{x}, \mathbf{r}) := (\mathbb{F}, 2^m, 2^n, \mathbf{A}, \mathbf{B}, \mathbf{C}), \quad \mathbf{A}_b = \mathbf{B}_b = 0, \quad \mathbf{C}_b = (r_1 - b_1) \cdot \mathbf{e}_0.$$

Both are deterministic and ignore their input except for $\mathbf{r}$, running in a time $O(2^{m+n} \log |\mathbb{F}|)$ that depends only on $\mathbb{F}, m, n$. Padding their code to this length, as noted after Definition 2.1, puts both in $\mathcal{F}_1$.

Falsity holds for every $\mathbf{x}$ and every $\mathbf{r}$: for $\mathbf{z}$ with $z_0 = 1$ the residual in row $\mathbf{b}$ is $(\mathbf{A}_b \mathbf{z})(\mathbf{B}_b \mathbf{z}) - \mathbf{C}_b \mathbf{z} = b_1 - r_1$, and the rows with $b_1 = 0$ and $b_1 = 1$ cannot both vanish.

Now suppose $\mathbf{x} \models \text{Bind}(\mathbf{x}, \mathbf{r})$, so the generator halts within budget, outputs exactly this R1CS instance, and its encoding has length at most T ; hence the verifier does not reject in step 1. Fix the honest prover's randomness, and let the later provers run it on $\mathbf{z}^\circ$, whose encoding f° is the message already sent. The initial sumcheck claim holds,

$$\sum_{\mathbf{b} \in \{0,1\}^m} \hat{G}(\mathbf{b}) = \sum_{\mathbf{b} \in \{0,1\}^m} \text{eq}(\mathbf{b}, \mathbf{r}) \cdot (b_1 - r_1) = r_1 - r_1 = 0,$$

by the multilinear interpolation identity. At the terminal point γ the honest prover supplies $\mu_{\mathbf{A}} = \mu_{\mathbf{B}} = 0$ and $\mu_{\mathbf{C}} = r_1 - \gamma_1$, and $\hat{A}(\gamma, \cdot) = \hat{B}(\gamma, \cdot) = 0$ while $\hat{C}(\gamma, \cdot) = (r_1 - \gamma_1)\mathbf{e}_0$. Hence, for every later challenge ε , the lincheck vector and target are $\mathbf{v} = (1 + \varepsilon^3(r_1 - \gamma_1))\mathbf{e}_0$ and $1 + \varepsilon^3\mu_{\mathbf{C}}$, so $\langle \mathbf{z}^\circ, \mathbf{v} \rangle$ equals the target and $f^\circ = \mathcal{C}(\mathbf{z}^\circ)$. Perfect completeness of the sumcheck and of the lincheck protocol makes the verifier accept, for every continuation of the challenges. $\square$

Corollary 6.6. *Under the hypotheses of Lemma 6.5, let $\theta_{\mathbf{V}}$ be the randomness degree of IOP_{ml} and assume $\theta_{\mathbf{V}}(1 + \theta_h) < \mathbf{c}$. Then for every pair of hash families $h = (h_{\text{MT}}, h_{\text{FS}})$ evaluable in time $O(L^{\theta_h})$ one can efficiently construct a false instance $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R}_{\text{R1CS}}[\mathbf{c}])$ and an argument π_h accepted by $\text{BCS}^h(\text{IOP}_{\text{ml}})$ with probability one.*

Proof. Fix θ with $\theta_{\mathbf{V}}(1 + \theta_h) \leq \theta < \mathbf{c}$. By Corollary 4.7, $\mathcal{S}$ is self-referential for $\mathcal{F}_\theta$, so IOP_{ml} is $(1, \theta, 1)$ -diagonalizable by Lemma 6.5. Apply Theorem 3.10. $\square$

The first malicious message is a fixed codeword, so the sharper Corollary 3.11 applies and removes the composition of degrees in Corollary 6.6.

Corollary 6.7. *Under the hypotheses of Lemma 6.5, let $h = (h_{\text{MT}}, h_{\text{FS}})$ be round-domain-separated hash families evaluable in time $O(L^{\theta_h})$ on inputs of length L , with $\max\{1, \theta_h\} < \mathbf{c}$. Then one can efficiently construct a false instance $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R}_{\text{R1CS}}[\mathbf{c}])$ and an argument π_h accepted by $\text{BCS}^h(\text{IOP}_{\text{ml}})$ with probability one.*

Proof. The algorithm $\tilde{\mathbf{P}}_1$ of Lemma 6.5 ignores its input, so its message $\mathcal{C}(\mathbf{e}_0)$ and root a_1° are constants. By Corollary 4.7 the scheme $\mathcal{S}$ is self-referential for $\mathcal{F}_\theta$ with $\theta := \max\{1, \theta_h\} < \mathbf{c}$, so IOP_{ml} is $(1, \theta, 1)$ -diagonalizable and Corollary 3.11 applies. $\square$

7 BCS attacks against univariate zerocheck

We give a univariate counterpart of Construction 6.2, using the univariate sumcheck identity that underlies protocols such as Aurora [BCRSVW19] and STIR [ACFY24]. For simplicity, we restrict to generated R1CS instances whose generated matrices are square, with $M = N = n$, and assume that $\mathbb{F}^*$ contains a multiplicative subgroup of order n .

7.1 Preliminaries

We first recall the polynomial evaluation relation.

Definition 7.1. Let $\mathcal{C}: \mathbb{F}^{<n}[X] \rightarrow \Sigma^\ell$ be an error-correcting code. The **polynomial evaluation relation** $\mathcal{R}_{\text{eval}}$ consists of tuples $(\mathbf{x}, \mathbf{y}, \mathbf{w})$ of the form

$$\mathbf{x} = \left(\alpha, (d_i)_{i \in [t]}, (\mu_i)_{i \in [t]} \right), \quad \mathbf{y} = (f_i)_{i \in [t]}, \quad \mathbf{w} = (\hat{f}_i)_{i \in [t]},$$

where $\alpha, \mu_i \in \mathbb{F}$, $d_i \leq n$, $f_i \in \Sigma^\ell$, and $\hat{f}_i \in \mathbb{F}^{<d_i}[X]$, such that, for every $i \in [t]$,

$$f_i = \mathcal{C}(\hat{f}_i) \quad \text{and} \quad \hat{f}_i(\alpha) = \mu_i.$$

For $\delta \in (0, 1)$, let $\mathcal{R}_{\text{eval}, \delta}$ be the δ -relaxation of $\mathcal{R}_{\text{eval}}$, obtained by replacing $f_i = \mathcal{C}(\hat{f}_i)$ with $\Delta(f_i, \mathcal{C}(\hat{f}_i)) \leq \delta$.

Fix a multiplicative subgroup $\mathcal{H} = \{1, \omega, \dots, \omega^{n-1}\} \subseteq \mathbb{F}^*$ of order n , and identify ω^i with coordinate $i \in \{0, \dots, n-1\}$. In particular, n is nonzero in $\mathbb{F}$. Let

$$Z_{\mathcal{H}}(X) := X^n - 1.$$

For $\rho \in \mathbb{F}$, let $P_\rho \in \mathbb{F}^{<n}[X]$ be the interpolation polynomial satisfying $P_\rho(\omega^i) = \rho^i$. For a matrix $\mathbf{M} \in \mathbb{F}^{n \times n}$, let $\hat{M}_\rho \in \mathbb{F}^{<n}[X]$ be the interpolation polynomial satisfying

$$\hat{M}_\rho(\omega^j) := \sum_{i=0}^{n-1} \rho^i M_{i,j}. \quad (9)$$

7.2 Construction

Construction 7.2 (Generated-R1CS univariate zerocheck). The public-coin IOP $\text{IOP}_{\text{uni}} = (\mathbf{P}, \mathbf{V})$ is defined as follows. Let $\mathcal{C}: \mathbb{F}^{<n}[X] \rightarrow \Sigma^\ell$ be an error-correcting code with polynomial block length and a polynomial-time encoder. Let $(\mathbf{P}_{\text{eval}}, \mathbf{V}_{\text{eval}})$ be an IOPP for $\mathcal{R}_{\text{eval}}$. Fix a budget exponent $\mathbf{c} \geq 2$; the protocol is for the generated R1CS relation $\mathcal{R}_{\text{R1CS}}[\mathbf{c}]$, restricted to instances whose generated matrices are over $\mathbb{F}$ with dimensions $n \times n$.

- **Inputs.** The verifier $\mathbf{V}$ receives $\mathbf{x} = (1^T, s)$. The honest prover $\mathbf{P}$ additionally receives $\mathbf{w} = \mathbf{z} \in \mathbb{F}^n$ such that $(\mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\text{R1CS}}[\mathbf{c}]$.
- **Interactive phase.**
 1. **Compute R1CS matrices.** The parties compute $\sigma := \mathfrak{R}_s(\epsilon)$. The verifier rejects unless the execution halts within $T^{\mathbf{c}}$ steps and σ is an R1CS instance $(\mathbb{F}, n, n, \mathbf{A}, \mathbf{B}, \mathbf{C})$ over $\mathbb{F}$ with three matrices in $\mathbb{F}^{n \times n}$, of encoding length $|\sigma| \leq T$.

2. **Witness polynomial oracles.** The prover sends

$$\pi_1 := (f_A, f_B, f_C, f_0, f_w) = (\mathcal{C}(\hat{f}_A), \mathcal{C}(\hat{f}_B), \mathcal{C}(\hat{f}_C), \mathcal{C}(\hat{f}_0), \mathcal{C}(\hat{f}_w)) ,$$

where

$$\hat{f}_A, \hat{f}_B, \hat{f}_C \in \mathbb{F}^{<n}[X] \quad \text{and} \quad \hat{f}_0, \hat{f}_w \in \mathbb{F}^{<n-1}[X] .$$

In the honest case, let $\hat{z} \in \mathbb{F}^{<n}[X]$ interpolate $\hat{z}(\omega^i) = z_i$, and set

$$\begin{aligned} \hat{f}_M(\omega^i) &:= (\mathbf{M}\mathbf{z})_i && \text{for } \mathbf{M} \in \{\mathbf{A}, \mathbf{B}, \mathbf{C}\} \text{ and } 0 \leq i < n, \\ \hat{f}_0(X) &:= \frac{\hat{f}_A(X)\hat{f}_B(X) - \hat{f}_C(X)}{Z_{\mathcal{H}}(X)} , \\ \hat{f}_w(X) &:= \frac{\hat{z}(X) - 1}{X - 1} . \end{aligned}$$

3. **Matrix batching randomness.** The verifier samples and sends $\rho \leftarrow \mathbb{F}$.

4. **Univariate sumcheck.** Define

$$\hat{f}_z(X) := (X - 1)\hat{f}_w(X) + 1 ,$$

and

$$\begin{aligned} \hat{u}(X) &:= P_\rho(X)\hat{f}_A(X) - \hat{A}_\rho(X)\hat{f}_z(X) \\ &\quad + \rho^n \left(P_\rho(X)\hat{f}_B(X) - \hat{B}_\rho(X)\hat{f}_z(X) \right) \\ &\quad + \rho^{2n} \left(P_\rho(X)\hat{f}_C(X) - \hat{C}_\rho(X)\hat{f}_z(X) \right) . \end{aligned} \tag{10}$$

The prover sends its second oracle

$$\pi_2 := (f_{g_1}, f_{g_2}) = (\mathcal{C}(\hat{g}_1), \mathcal{C}(\hat{g}_2)) ,$$

where $\hat{g}_1, \hat{g}_2 \in \mathbb{F}^{<n-1}[X]$. In the honest case, they are the unique polynomials satisfying

$$\hat{u}(X) = Z_{\mathcal{H}}(X)\hat{g}_1(X) + X\hat{g}_2(X) .$$

To see existence, honest data satisfy $\sum_{x \in \mathcal{H}} \hat{u}(x) = 0$. Since n is nonzero in $\mathbb{F}$ and $\deg \hat{u} \leq 2n - 2$, this is equivalent to $[X^0]\hat{u} + [X^n]\hat{u} = 0$. Hence the remainder after division by $Z_{\mathcal{H}}$ has zero constant term and is divisible by X . Polynomial division also proves uniqueness and the stated degree bounds.

5. **Evaluation randomness.** The verifier samples and sends $\alpha \leftarrow \mathbb{F}$.

6. **Polynomial evaluation proof.** In the first message of the polynomial evaluation proof, the prover includes the claims

$$\boldsymbol{\mu} := (\mu_A, \mu_B, \mu_C, \mu_0, \mu_w, \mu_{g_1}, \mu_{g_2}) .$$

The parties then run $(\mathbf{P}_{\text{eval}}, \mathbf{V}_{\text{eval}})$ on the polynomial evaluation instance

$$\begin{aligned} \mathbf{x}_{\text{eval}} &= (\alpha, (n, n, n, n - 1, n - 1, n - 1, n - 1), \boldsymbol{\mu}) , \\ \mathbf{y}_{\text{eval}} &= (f_A, f_B, f_C, f_0, f_w, f_{g_1}, f_{g_2}) . \end{aligned}$$

- **Decision phase.** The verifier reconstructs

$$\mu_z := (\alpha - 1)\mu_w + 1,$$

and evaluates (10) at α by replacing the prover polynomials with their claimed evaluations. It accepts if and only if the generator terminates correctly, the polynomial evaluation proof accepts, and

$$\begin{aligned}\mu_A \mu_B - \mu_C &= Z_{\mathcal{H}}(\alpha) \mu_0, \\ \hat{u}(\alpha) &= Z_{\mathcal{H}}(\alpha) \mu_{g_1} + \alpha \mu_{g_2}.\end{aligned}$$

7.3 Soundness proof

Write k_{eval} for the number of rounds of the polynomial evaluation proof, and let $(\varepsilon_{\text{eval},j}(\delta))_{j \in [k_{\text{eval}}]}$ be its round-by-round knowledge errors for $\mathcal{R}_{\text{eval},\delta}$. The proof of the following theorem is almost identical to that in prior work [ACFY24].⁷

Theorem 7.3. *For every $\delta \in (0, 1)$ such that $(\mathbf{P}_{\text{eval}}, \mathbf{V}_{\text{eval}})$ has round-by-round knowledge soundness for $\mathcal{R}_{\text{eval},\delta}$ with polynomial extraction time, the protocol IOP_{uni} has round-by-round knowledge soundness for $\mathcal{R}_{\text{RCS}}[c]$, with polynomial extraction time and round errors*

$$\left(\frac{(3n-1) |\Lambda(\mathcal{C}, \delta)|^5}{|\mathbb{F}|}, \frac{(2n-2) |\Lambda(\mathcal{C}, \delta)|^7}{|\mathbb{F}|}, (\varepsilon_{\text{eval},j}(\delta))_{j \in [k_{\text{eval}}]} \right).$$

In particular, the error in every round is at most

$$\varepsilon_{\text{uni}}(\delta) := \max \left\{ \frac{(3n-1) |\Lambda(\mathcal{C}, \delta)|^5}{|\mathbb{F}|}, \frac{(2n-2) |\Lambda(\mathcal{C}, \delta)|^7}{|\mathbb{F}|}, \max_{j \in [k_{\text{eval}}]} \varepsilon_{\text{eval},j}(\delta) \right\}. \quad (11)$$

Combining Theorem 7.3 with the BCS knowledge-soundness theorem gives the soundness of the compiled protocol.

Corollary 7.4 (Soundness of the compiled protocol). *Under the hypotheses of Theorem 7.3, let $k_{\text{uni}} = 2 + k_{\text{eval}}$ be the number of rounds of IOP_{uni} . In the random oracle model, for every malicious argument prover making at most q random-oracle queries, there is an adaptive straight-line extractor for $\text{BCS}^h(\text{IOP}_{\text{uni}})$ whose knowledge error for $\mathcal{R}_{\text{RCS}}[c]$ is at most*

$$(q + k_{\text{uni}}) \cdot \varepsilon_{\text{uni}}(\delta) + \varepsilon_{\text{BCS}} \left(\lambda, q, (l_i)_{i \in [k_{\text{uni}}]} \right).$$

Proof. Apply Theorem 2.15 with the uniform round error from (11). □

7.4 Attack

As in Section 6, the attack is an instance of Theorem 3.10 with $i = 1$: only the matrix batching randomness ρ must be controlled. Write $\mathcal{S}$ for the semantic scheme of Corollary 4.7, now for the field $\mathbb{F}$ and dimensions $n \times n$.

⁷Better bounds can be obtained by (for example) out-of-domain sampling [BGKS20] and analyzing the oracles as interleaved codes. For simplicity, we do not perform these optimizations (as they do not affect the attacks).

Lemma 7.5. *Assume $n \geq 2$, and that the polynomial evaluation proof has an efficient prover and perfect completeness. Then IOP_{uni} is $(1, 1)$ -programmable relative to $\mathcal{S}$.*

Proof. Let $\mathbf{z}^\circ := \mathbf{e}_0$ with interpolation $\hat{\mathbf{z}}^\circ \in \mathbb{F}^{<n}[X]$, and let $\tilde{\mathbf{P}}_1$ ignore its input and output the componentwise encoding π_1° of

$$\left(\hat{f}_A^\circ, \hat{f}_B^\circ, \hat{f}_C^\circ, \hat{f}_0^\circ, \hat{f}_w^\circ\right) := \left(0, 0, 0, 0, \frac{\hat{\mathbf{z}}^\circ(X)-1}{X-1}\right),$$

which meets all declared degree bounds. Let $\text{Bind}(\mathbf{x}, \rho)$ output the R1CS instance $(\mathbb{F}, n, n, \mathbf{A}, \mathbf{B}, \mathbf{C})$ with

$$\mathbf{A}_i = \mathbf{B}_i = 0 \quad (0 \leq i < n), \quad \mathbf{C}_0 = -\rho \cdot \mathbf{e}_0, \quad \mathbf{C}_1 = \mathbf{e}_0, \quad \mathbf{C}_i = 0 \quad (2 \leq i < n).$$

As in Lemma 6.5, both are deterministic and ignore their input except for ρ , running in a time $O(n^2 \log |\mathbb{F}|)$ that depends only on $\mathbb{F}, n$; padding puts both in $\mathcal{F}_1$.

Falsity holds for every $\mathbf{x}$ and every ρ , and does not even depend on ρ : a witness would have $z_0 = 1$, whereas row 1 demands $0 = \mathbf{C}_1 \mathbf{z} = z_0$.

Now suppose $\mathbf{x} \models \text{Bind}(\mathbf{x}, \rho)$, so the generator halts within budget, outputs exactly this R1CS instance, and its encoding has length at most T ; hence the verifier accepts the generator's output. Then $\hat{A}_\rho = \hat{B}_\rho = 0$, and for every $0 \leq j < n$ (9) gives $\hat{C}_\rho(\omega^j) = -\rho(\mathbf{e}_0)_j + \rho(\mathbf{e}_0)_j = 0$, so $\hat{C}_\rho \equiv 0$. As $\hat{f}_A^\circ, \hat{f}_B^\circ, \hat{f}_C^\circ$ vanish, (10) gives $\hat{u} \equiv 0$. Let $\tilde{\mathbf{P}}_2$ send $\pi_2^\circ := (\mathcal{C}(0), \mathcal{C}(0))$ and, at any evaluation challenge α , let the remaining provers claim $\boldsymbol{\mu}^\circ = (0, 0, 0, 0, \hat{f}_w^\circ(\alpha), 0, 0)$ and run the honest polynomial evaluation prover with fixed randomness. The two decision equations then read $0 = 0$, and perfect completeness of the evaluation proof makes the verifier accept, for every continuation of the challenges. $\square$

Corollary 7.6. *Under the hypotheses of Lemma 7.5, let $\theta_{\mathbf{V}}$ be the randomness degree of IOP_{uni} and assume $\theta_{\mathbf{V}}(1 + \theta_h) < \mathbf{c}$. Then for every pair of hash families $h = (h_{\text{MT}}, h_{\text{FS}})$ evaluable in time $O(L^{\theta_h})$ one can efficiently construct a false instance $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R}_{\text{R1CS}}[\mathbf{c}])$ and an argument π_h accepted by $\text{BCS}^h(\text{IOP}_{\text{uni}})$ with probability one.*

Proof. As for Corollary 6.6, using Lemma 7.5. $\square$

Corollary 7.7. *Under the hypotheses of Lemma 7.5, let $h = (h_{\text{MT}}, h_{\text{FS}})$ be round-domain-separated hash families evaluable in time $O(L^{\theta_h})$ on inputs of length L , with $\max\{1, \theta_h\} < \mathbf{c}$. Then one can efficiently construct a false instance $\mathbf{x}_h \notin \mathcal{L}(\mathcal{R}_{\text{R1CS}}[\mathbf{c}])$ and an argument π_h accepted by $\text{BCS}^h(\text{IOP}_{\text{uni}})$ with probability one.*

Proof. As for Corollary 6.7: the message π_1° of Lemma 7.5 is a constant, so Corollary 3.11 applies with $\theta' = 1$ and $\theta = \max\{1, \theta_h\} < \mathbf{c}$, which is admissible by Corollary 4.7. $\square$

Acknowledgements

We thank Ron Rothblum for insightful discussions on [KRS25] and this work. We thank Dan Boneh and Eran Tromer for helpful comments on presentation.

The author is supported in part by the Ethereum Foundation.

A Attack generator

We present a concrete Python program instantiating F_h of Corollary 6.7 for $m = n = 1$, using SHA-256. It is a quine: TEMPLATE reconstructs the complete program as SOURCE, which is the code s of the instance $\mathbf{x} = (1^T, s)$ whose identifier is hashed, together with the Merkle root a_1° of $\mathcal{C}(e_0)$, to obtain r_1 ; here $\mathcal{C}$ is the rate- $1/2$ Reed–Solomon code evaluating the witness, read as a polynomial of degree below 2, on $\{1, 2, 3, 4\}$. The generator then outputs the R1CS instance $(\mathbb{F}, 2, 2, \mathbf{A}, \mathbf{B}, \mathbf{C})$ with $\mathbf{A} = \mathbf{B} = \mathbf{0}$ and $\mathbf{C}_b = (r_1 - b_1)e_0$, over the field $\mathbb{F} = \mathbb{Z}/(2^{127} - 1)\mathbb{Z}$. The clock is fixed to $T = 4096$: its encoded output is 210 bytes long, so $|\sigma| \leq T$ as Definition 4.2 requires, and the few hash evaluations it performs are well within T^c steps already for $c = 2$.

From the repository root, verify the quine and run the generator with

```
python3 poc.py | cmp - poc.py
python3 poc.py --run
```

The first command produces no output on success; the second prints the first oracle $\mathcal{C}(e_0)$, the challenge r_1 , and the generated instance. Only the diagonalization is implemented: the sumcheck and the proximity test of Construction 6.2 play no role in the attack and are omitted.

```
1 import hashlib
2 import json
3 import sys
4
5 TEMPLATE = 'import hashlib\nimport json\nimport sys\n\nTEMPLATE = %r\nSOURCE = TEMPLATE %%
    TEMPLATE\n\nFIELD = 2**127 - 1\nROWS = COLUMNS = 2\nDOMAIN = (1, 2, 3, 4)\nCLOCK = 4096\n\n
    \ndef encode_field(value):\n    return (value %% FIELD).to_bytes(16, "big")\n\n\ndef
    hash_bytes(*parts):\n    digest = hashlib.sha256()\n    for part in parts:\n        digest.
    update(len(part).to_bytes(8, "big") + part)\n    return digest.digest()\n\n\ndef rs_encode(
    message):\n    """Rate-1/2 Reed--Solomon encoding of a witness in FIELD^COLUMNS."""\n
    return [sum(c * pow(x, i, FIELD) for i, c in enumerate(message)) %% FIELD\n            for
    x in DOMAIN]\n\n\ndef merkle_root(codeword):\n    leaves = [hash_bytes(b"merkle/witness",
    encode_field(value))\n                    for value in codeword]\n    return hash_bytes(b"merkle/
    witness", *leaves)\n\n\ndef encode_instance(matrices):\n    parts = [FIELD.to_bytes(16, "
    big"), bytes([ROWS, COLUMNS])]\n    for matrix in matrices:\n        for row in matrix:\n
            for value in row:\n                parts.append(encode_field(value))\n    return
    b"".join(parts)\n\n\ndef statement_id():\n    return hash_bytes(b"generated-riCS", b"1" *
    CLOCK, SOURCE.encode("utf-8"))\n\n\ndef generate():\n    commitment = merkle_root(rs_encode
    ([1, 0]))\n    digest = hash_bytes(b"fs/zerocheck", statement_id(), commitment)\n    r1 =
    int.from_bytes(digest, "big") %% FIELD\n    zero = [[0] * COLUMNS for _ in range(ROWS)]\n
    c_matrix = [[r1, 0], [(r1 - 1) %% FIELD, 0]]\n    return r1, (zero, zero, c_matrix)\n\n
    \ndef main():\n    if "--run" not in sys.argv:\n        print(SOURCE, end="")\n
    return\n    r1, matrices = generate()\n    encoding = encode_instance(matrices)\n    assert
    len(encoding) <= CLOCK, "instance does not fit the clock"\n    a_matrix, b_matrix,
    c_matrix = matrices\n    print(json.dumps({\n        "clock": CLOCK,\n        "codeword":
    rs_encode([1, 0]),\n        "r1": r1,\n        "field": FIELD,\n        "rows": ROWS,\n
        "columns": COLUMNS,\n        "A": a_matrix,\n        "B": b_matrix,\n        "C":
    c_matrix,\n        "encoding_bytes": len(encoding),\n    }, indent=2))\n\n\nif __name__ ==
    "__main__":\n    main()\n'
```

```
6 SOURCE = TEMPLATE % TEMPLATE
7
8 FIELD = 2**127 - 1
9 ROWS = COLUMNS = 2
10 DOMAIN = (1, 2, 3, 4)
11 CLOCK = 4096
```

```

12
13
14 def encode_field(value):
15     return (value % FIELD).to_bytes(16, "big")
16
17
18 def hash_bytes(*parts):
19     digest = hashlib.sha256()
20     for part in parts:
21         digest.update(len(part).to_bytes(8, "big") + part)
22     return digest.digest()
23
24
25 def rs_encode(message):
26     """Rate-1/2 Reed--Solomon encoding of a witness in  $\text{FIELD}^{\text{COLUMNS}}$ ."""
27     return [sum(c * pow(x, i, FIELD) for i, c in enumerate(message)) % FIELD
28             for x in DOMAIN]
29
30
31 def merkle_root(codeword):
32     leaves = [hash_bytes(b"merkle/witness", encode_field(value))
33               for value in codeword]
34     return hash_bytes(b"merkle/witness", *leaves)
35
36
37 def encode_instance(matrices):
38     parts = [FIELD.to_bytes(16, "big"), bytes([ROWS, COLUMNS])]
39     for matrix in matrices:
40         for row in matrix:
41             for value in row:
42                 parts.append(encode_field(value))
43     return b"".join(parts)
44
45
46 def statement_id():
47     return hash_bytes(b"generated-ribs", b"1" * CLOCK, SOURCE.encode("utf-8"))
48
49
50 def generate():
51     commitment = merkle_root(rs_encode([1, 0]))
52     digest = hash_bytes(b"fs/zerocheck", statement_id(), commitment)
53     r1 = int.from_bytes(digest, "big") % FIELD
54     zero = [[0] * COLUMNS for _ in range(ROWS)]
55     c_matrix = [[r1, 0], [(r1 - 1) % FIELD, 0]]
56     return r1, (zero, zero, c_matrix)
57
58
59 def main():
60     if "--run" not in sys.argv:
61         print(SOURCE, end="")
62         return
63     r1, matrices = generate()
64     encoding = encode_instance(matrices)
65     assert len(encoding) <= CLOCK, "instance does not fit the clock"
66     a_matrix, b_matrix, c_matrix = matrices
67     print(json.dumps({

```

```

68     "clock": CLOCK,
69     "codeword": rs_encode([1, 0]),
70     "r1": r1,
71     "field": FIELD,
72     "rows": ROWS,
73     "columns": COLUMNS,
74     "A": a_matrix,
75     "B": b_matrix,
76     "C": c_matrix,
77     "encoding_bytes": len(encoding),
78 }, indent=2))
79
80
81 if __name__ == "__main__":
82     main()

```

Listing 1: Quine implementing the generator of Corollary 6.7.

References

- [ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “STIR: Reed–Solomon Proximity Testing with Fewer Queries”. In: *Proceedings of the 44th Annual International Cryptology Conference*. CRYPTO ’24. 2024, pp. 380–413.
- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. “WHIR: Reed–Solomon Proximity Testing with Super-Fast Verification”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [AY25] Gal Arnon and Eylon Yogev. “Towards a White-Box Secure Fiat-Shamir Transformation”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025.
- [BBHMR19] James Bartusek, Liron Bronfman, Justin Holmgren, Fermi Ma, and Ron D. Rothblum. “On the (In)security of Kilian-Based SNARGs”. In: *Theory of Cryptography – 17th International Conference, TCC 2019, Nuremberg, Germany, December 1–5, 2019, Proceedings, Part II*. Ed. by Dennis Hofheinz and Alon Rosen. Vol. 11892. Lecture Notes in Computer Science. Springer, 2019, pp. 522–551. DOI: 10.1007/978-3-030-36033-7_20.
- [BCFRRZ25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. “Blaze: Fast SNARKs from Interleaved RAA Codes”. In: *Proceedings of the 44th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’25. 2025.
- [BCFW25] Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. “Linear-Time Accumulation Schemes”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 369–399.
- [BCRSVW19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. “Aurora: Transparent Succinct Arguments for R1CS”. In: *Proceedings of the 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’19. 2019, pp. 103–128.
- [BCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. “Interactive Oracle Proofs”. In: *Proceedings of the 14th Theory of Cryptography Conference*. TCC ’16-B. 2016, pp. 31–60.

- [BFRW26] Benedikt Bünz, Giacomo Fenzi, Ron D. Rothblum, and William Wang. “TensorSwitch: Nearly Optimal Polynomial Commitments from Tensor Codes”. In: *Proceedings of the 46th Annual International Cryptology Conference*. CRYPTO ’26. 2026, pp. 297–329.
- [BGKS20] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. “DEEP-FRI: Sampling Outside the Box Improves Soundness”. In: *Proceedings of the 11th Innovations in Theoretical Computer Science Conference*. ITCS ’20. 2020, 5:1–5:32.
- [BHK26] Chris Brzuska, Pavel Hubáček, and Aleksa Kalsta. “Simple Attacks against (Extended) Fiat-Shamir”. In: *Proceedings of the 29th IACR International Conference on Practice and Theory of Public-Key Cryptography*. PKC ’26. 2026, pp. 510–543.
- [BR93] Mihir Bellare and Phillip Rogaway. “Random Oracles are Practical: A Paradigm for Designing Efficient Protocols”. In: *Proceedings of the 1st ACM Conference on Computer and Communications Security*. CCS ’93. 1993, pp. 62–73.
- [BRW26] Benedikt Bünz, Ron D. Rothblum, and William Wang. *Flock: Fast Proving for Batch Boolean Computations*. Cryptology ePrint Archive, Paper 2026/1329. 2026. URL: <https://eprint.iacr.org/2026/1329>.
- [Bar01] Boaz Barak. “How to Go Beyond the Black-Box Simulation Barrier”. In: *Proceedings of the 42nd IEEE Symposium on Foundations of Computer Science*. FOCS ’11. 2001, pp. 106–115.
- [CCHLRR18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. *Fiat-Shamir From Simpler Assumptions*. Cryptology ePrint Archive, Report 2018/1004. 2018.
- [CGH04] Ran Canetti, Oded Goldreich, and Shai Halevi. “The random oracle methodology, revisited”. In: *Journal of the ACM* 51.4 (2004), pp. 557–594.
- [CO25] Alessandro Chiesa and Michele Orrù. “A Fiat-Shamir Transformation From Duplex Sponges”. In: *Proceedings of the 23rd Theory of Cryptography Conference*. TCC ’25. 2025, pp. 452–474.
- [CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024. URL: <https://github.com/hash-based-snargs-book>.
- [DMWG23] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. “Weak Fiat-Shamir Attacks on Modern Proof Systems”. In: *2023 IEEE Symposium on Security and Privacy*. IEEE, 2023, pp. 199–216. DOI: 10.1109/SP46215.2023.10179408.
- [DP26] Benjamin Diamond and Jim Posen. “Polylogarithmic Proofs for Multilinears over Binary Towers”. In: *Proceedings of the 45th Annual International Conference on the Theory and Applications of Cryptographic Techniques*. EUROCRYPT ’26. 2026, pp. 3–32.
- [FS86] Amos Fiat and Adi Shamir. “How to prove yourself: practical solutions to identification and signature problems”. In: *Proceedings of the 6th Annual International Cryptology Conference*. CRYPTO ’86. 1986, pp. 186–194.
- [Fis05] Marc Fischlin. “Communication-Efficient Non-interactive Proofs of Knowledge with On-line Extractors”. In: *Proceedings of the 25th Annual International Cryptology Conference*. CRYPTO ’05. 2005, pp. 152–168.
- [GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. “Quadratic Span Programs and Succinct NIZKs without PCPs”. In: *Proceedings of the 32nd Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’13. 2013, pp. 626–645.
- [GK03] Shafi Goldwasser and Yael Tauman Kalai. “On the (In)security of the Fiat-Shamir Paradigm”. In: *Proceedings of the 44th Annual IEEE Symposium on Foundations of Computer Science*. FOCS ’03. 2003, pp. 102–113.

- [GKR15] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. “Delegating Computation: Interactive Proofs for Muggles”. In: *Journal of the ACM* 62.4 (2015), 27:1–27:64.
- [KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. “How to Prove False Statements: Practical Attacks on Fiat-Shamir”. In: *Proceedings of the 45th Annual International Cryptology Conference*. CRYPTO ’25. 2025, pp. 3–26.
- [KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. “Constant-Size Commitments to Polynomials and Their Applications”. In: *Proceedings of the 16th International Conference on the Theory and Application of Cryptology and Information Security*. ASIACRYPT ’10. 2010, pp. 177–194.
- [Kle52] Stephen Cole Kleene. *Introduction to Metamathematics*. Amsterdam: North-Holland Publishing Company, 1952.
- [Mic00] Silvio Micali. “Computationally Sound Proofs”. In: *SIAM Journal on Computing* 30.4 (2000). Preliminary version appeared in FOCS ’94., pp. 1253–1298.
- [NA25] Andrija Novakovic and Guillermo Angeris. *Ligerito: A Small and Concretely Fast Polynomial Commitment Scheme*. Cryptology ePrint Archive, Report 2025/1187. 2025.
- [PS96] David Pointcheval and Jacques Stern. “Security Proofs for Signature Schemes”. In: *Proceedings of the 14th Annual International Conference on Theory and Application of Cryptographic Techniques*. EUROCRYPT ’96. 1996, pp. 387–398.
- [Pol24] Polyhedra Network. *Expander*. <https://github.com/PolyhedraZK/Expander>. Accessed: 2025-01-15. 2024.
- [RRR16] Omer Reingold, Ron Rothblum, and Guy Rothblum. “Constant-Round Interactive Proofs for Delegating Computation”. In: *Proceedings of the 48th ACM Symposium on the Theory of Computing*. STOC ’16. 2016, pp. 49–62.
- [Set20] Srinath Setty. “Spartan: Efficient and general-purpose zkSNARKs without trusted setup”. In: *Proceedings of the 40th Annual International Cryptology Conference*. CRYPTO ’20. 2020, pp. 704–737.
- [ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. “BaseFold: Efficient Field-Agnostic Polynomial Commitment Schemes from Foldable Codes”. In: *Proceedings of the 44th Annual International Cryptology Conference*. Vol. 14929. CRYPTO ’24. 2024, pp. 138–169.